[Project Name TBD]: A Privacy-First Operating System with Integrated Knowledge Infrastructure

Design, Architecture, and Philosophy

Tim[a]

a University of Innsbruck, Austria

Abstract TODO

Keywords operating system, knowledge graph, privacy, user sovereignty, AI integration, Linux

The Art, Science, and Engineering of Programming

Perspective The Engineering of Programming

Area of Submission Programming environments, Operating systems, Social coding

© Tim
This work is licensed under a "CC BY 4.0" license.
Submitted to *The Art, Science, and Engineering of Programming*.

1 Introduction

1.1 Motivation

Most Linux distributions are either conservative remixes of existing systems or highly opinionated tools for a specific niche. Neither describes this project.

The dominant desktop operating systems have a structural problem. Windows works against its own users: forced updates, advertisements in the start menu, and background telemetry (automatic reporting of usage data to the vendor) that nobody enabled. macOS is polished but locks users into an ecosystem designed to keep them paying and dependent. Every Linux distribution, however technically excellent, has failed to make the jump to people who simply want to use a computer without becoming a system administrator. The gap between "technically capable" and "actually usable" has not closed in thirty years.

This project exists to close that gap. It is a Linux-based operating system built from scratch with a different set of priorities: one that integrates a system-wide knowledge graph, AI interfaces, and modern tooling not as features bolted on top, but as foundational infrastructure. It takes a clear stance on user sovereignty, European software independence, and not being annoying to use. If an operating system is being built in 2025 anyway, there is no reason to repeat patterns from 1995.

1.2 Target Audience

The primary audience is people switching from macOS or Windows who want a genuinely good alternative: modern in appearance, functional out of the box, and respectful of their autonomy. Consumer-first, not enterprise-first. The macOS user tired of lock-in but unwilling to compromise on quality. The Windows user whose computer feels like it belongs to Microsoft more than to them. They do not want to configure. They want to arrive and work, and feel like the operating system is actually theirs.

Business deployment is a future consideration. The architecture supports managed environments and administrator policies. However, one line does not move: an administrator can configure defaults and restrict software; an administrator cannot surveil users or override their personal data ownership. Company needs will never come at the expense of user rights.

1.3 Design Principles

The following principles govern every design decision in this project. They are not aspirational marketing language; they are constraints that break ties and override convenience.

For the user, not for the platform. No advertisements, no tracking, no dark patterns, no telemetry without explicit consent. When a conflict arises between the user's interest

and anything else, the user wins. This applies to business deployments as well: admin policies configure the environment, they do not strip users of their data or autonomy.

Independence by default. No mandatory cloud accounts. No dependency on services that can be switched off by a company in another jurisdiction. The system works fully offline, self-hosted, and without phoning home. Cloud features are opt-in extensions, not prerequisites. This is a technical commitment, not a marketing label.

The knowledge graph belongs to you. All system intelligence runs locally by default. The knowledge graph, AI context, and activity history do not leave the machine without an explicit user action. Cloud AI providers are opt-in, configured by the user, and can be removed entirely.

Quality of life over feature count. Small things matter. A system that warns you when something is quietly invading your privacy, that remembers what you were working on, that switches themes system-wide in one action: these are not large features. They are the difference between a system that feels alive and one that feels like infrastructure. Thoughtful small features beat impressive large ones that nobody uses.

Out of the box, not out of patience. Everything works on first boot. No hunting for drivers, no configuring package managers, no reading wiki pages to get audio working. The standard use case requires zero setup. Advanced configuration is always available, never required.

Future-oriented, not hype-driven. New technology gets used when it makes sense, not because it is trending. AI integration and the knowledge graph are here because they genuinely change what a personal computer can do. If a system is being built from scratch, there is no excuse for dragging along outdated patterns because everyone else does.

Good-looking is not optional. A poor interface is a barrier to entry. The system must look better than what users are coming from, on first boot. Visual quality is not vanity; it is respect for the user's time and trust.

Coherent, not cobbled together. Shared configuration standards, shared theming, shared interfaces across all system components. Change the accent color once and it propagates everywhere. No component that looks like it was designed in a different decade by a different team.

Pragmatic, not ideological. No purity tests. The goal is a useful, trustworthy system. That said: no advertisements, no tracking, no dark patterns. Not because of ideology, but because that is bad product design.

Bridges, not walls. Where established standards and existing implementations exist, compatibility bridges are built rather than replacements. Apps built for other Linux

desktops run here. Standard Linux tools and hardware interfaces are supported. Nothing is deliberately broken for the sake of purity.

Transparent by default. The system can explain what it is doing and why, in plain language. Not buried in logs, but surfaced where the user can see and act on it. A system legible to its user is a system the user can trust.

1.4 Positioning

Figure 1 illustrates the design space this project occupies relative to existing systems. The goal is the upper-right quadrant: high usability *and* high user sovereignty. This combination does not currently exist in any mainstream desktop operating system.

Figure 1 Design space: ease of use versus user sovereignty. Existing mainstream systems occupy either the lower-right (usable but extractive) or upper-left (sovereign but inaccessible) quadrants. This project targets the upper-right.

1.5 What This Project Is Not

Clarifying what this project is not is as important as describing what it is.

It is not a minimal system stripped to its essentials. There is real complexity here, and that is intentional: a system that works for non-technical users requires more infrastructure, not less.

It is not a fully declarative system where every piece of software is managed through configuration files. Reproducibility is a goal; mandatory declarative configuration is not.

It is not a distro aimed at advanced Linux users only. New users are explicitly part of the target. Advanced users are welcome and well-served, but they are not the primary design constraint.

It is not a corporate product that happens to be open source. Business use is secondary; user rights are not negotiable.

It is not a Windows or macOS clone. The goal is a better alternative, not a familiar imitation.

2 System Overview

2.1 Architecture at a Glance

The system is organized as a vertical stack of layers. Lower layers provide services to higher layers; no layer reaches past its immediate neighbor. A problem in one layer does not propagate downward, and the upper layers can be developed, replaced, or disabled independently of the foundation beneath them. The AI layer, for example, sits at the top of the stack and can be removed entirely without affecting anything below it.

Figure 2 shows the full layer structure.

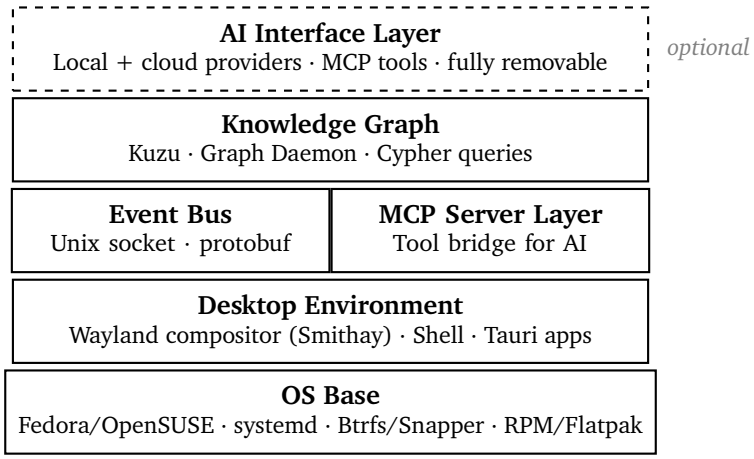

Figure 2 System layer structure. Each layer depends on the layer below it. The AI Interface Layer is dashed to indicate it is fully optional and can be disabled without affecting any other layer.

2.2 Layer Responsibilities

OS Base. The foundation is a standard Linux distribution providing the kernel, init system, package manager, and base userspace tools. Nothing at this layer is custom-built. The system builds on top of it; it does not replace it. Details are covered in Section 3.

Desktop Environment. The Wayland compositor and the shell sit at this layer. This is what the user directly interacts with: windows, the taskbar, Spotlight, notifications, the lock screen. UI applications are built with *Tauri* (a framework combining a Rust

backend with a web-based frontend), which gives them consistent access to system services without the memory overhead of Electron-style approaches.

Event Bus. The internal message bus handles all communication between custom system components. It receives, validates, and routes structured events from kernel monitoring, the display system, and first-party applications. It runs as a background service over Unix sockets using *Protocol Buffers* (protobuf) for schema-enforced message serialization. Its role is purely to move data; it does not interpret or store anything.

MCP Server Layer. The *Model Context Protocol* (MCP) is an open standard for exposing system capabilities as callable tools that AI models can invoke. This layer wraps system services in MCP-compatible interfaces, which gives the AI layer a structured, permission-gated way to act on the system without direct access to internals. The same interfaces are available to any MCP-compatible client.

Knowledge Graph. The graph database is the system's long-term memory. It stores events arriving from the Event Bus as typed nodes and relationships, making it possible to query across time, application boundaries, and data types. The Graph Daemon is the sole reader and writer of the underlying Kuzu database; no other component accesses the database file directly. Queries are expressed in Cypher, a declarative graph query language that reads similarly to natural language.

AI Interface Layer. The top layer provides AI capabilities to the rest of the system. It queries the Knowledge Graph for context, exposes tools via the MCP layer, and mediates between user requests and AI model providers. All AI providers, whether local models or remote API services, are interchangeable behind a common interface. This layer is entirely optional: disabling it removes AI features and nothing else. The system is fully functional without it.

2.3 Component Boundaries

The system's components divide cleanly into two categories: background services that run silently at all times, and applications with a user interface.

Background services are called *daemons* in Linux terminology: programs with no visible window that start at boot, wait to be called, and restart automatically if they crash. All daemons in this system are written in Rust for memory safety and predictable performance.

User-facing applications are built with Tauri, giving them a native Rust backend that communicates with daemons over Unix sockets, and a web-based frontend for the interface. This combination avoids the weight of Electron while still enabling a modern, consistent UI.

Table 1 Background services (daemons)

Service	Responsibility
Event Bus	Receives, validates, and routes internal events between components
Graph Daemon	Sole reader/writer of the Knowledge Graph database
Account Daemon	Credential management and login token handling
AI Layer Daemon	Runs AI models, coordinates tools, queries the graph
Module Runtime	Manages the lifecycle of shell extensions
Transfer Daemon	Controls data movement between user profiles
Compositor	Draws windows, manages the display (Wayland compositor)

Table 2 User-facing applications

Application	Notes
Shell	Taskbar, Spotlight, notifications, lock screen
Settings	System configuration UI
File Manager	First-party file browser
Store	App store UI
Terminal	First-party terminal emulator
Wine Manager	Windows compatibility management UI
Modules	Extensions running in isolated containers under Module Runtime

2.4 Communication Model

All inter-component communication uses *Unix sockets*: a fast, local-only channel built into the kernel. Unix sockets are not networked; they exist only on the local machine. Latency is measured in microseconds. Data transmitted over these sockets is serialized with Protocol Buffers, a compact binary format that enforces a strict schema. A message that does not match the expected schema is rejected before it reaches the handler.

The four communication paths in the system are:

Table 3 Inter-component communication paths

Path	Transport
Daemon ⟷ Daemon	Unix socket (protobuf)
App backend ⟷ Daemon	Unix socket (protobuf)
App backend ⟷ App frontend	Tauri command bridge (typed)
Compositor ⟷ Shell	Wayland protocols + Unix socket

A concrete example: a user action in the UI that queries the Knowledge Graph follows this path:

UI event → App backend → Unix socket → Graph Daemon → Database

Four steps, all local to the machine. The full round-trip is imperceptible to the user.

2.5 D-Bus Coexistence

D-Bus is the standard inter-process communication system used by nearly every Linux distribution. Network management, Bluetooth, hardware event notifications, and desktop portals all speak D-Bus. It is part of the Freedesktop standard and cannot be removed without breaking compatibility with the broader Linux ecosystem.

This system does not replace D-Bus. It runs alongside it. D-Bus remains responsible for everything it already handles: third-party app notifications, hardware events, portal interfaces for sandboxed apps. The custom Event Bus handles communication between project-internal components only. The two systems are parallel and non-overlapping.

This distinction matters for developers: if a component is part of this project, it communicates over the internal Event Bus. If a component is a standard Linux service or a third-party app, it communicates over D-Bus as it always has.

3 OS Foundation

Every operating system needs a foundation: a kernel that talks directly to hardware, a startup system, a package manager, and a base set of system tools. Building all of this from scratch would take decades and produce something inferior to what already exists. This project therefore builds on top of an existing Linux distribution that provides the entire foundation layer, and adds everything above it: the desktop environment, event system, knowledge graph, and AI layer. The distribution provides the load-bearing walls; the layout and everything inside them is custom.

3.1 Base Distribution

The base distribution has not yet been finalized; two candidates are under evaluation and the decision will be made before Phase 1 begins. Both use the *RPM* package format, which means the build pipeline and tooling are compatible with either choice.

The requirements for the base are deliberately narrow. Packages need to be current enough for modern development toolchains and recent hardware support, but not so fresh that unvalidated changes arrive daily. Governance should not be fully concentrated in a single commercial entity. *Btrfs* must be available as a first-class filesystem, since the update strategy described in Section 3.2 depends on it. Driver and package coverage must be broad enough to support consumer hardware without manual intervention.

Candidate A: Fedora. Fedora is a community distribution backed by Red Hat infrastructure but governed independently through the Fedora Council, which makes binding decisions on project direction without requiring Red Hat sign-off. Packages track upstream closely and a new release ships every six months, making it one of the more current stable distributions available. Btrfs has been the default filesystem since

Fedora 33, and the Flatpak and Wayland ecosystems are better supported here than on most other distributions.

Table 4 Fedora: evaluation summary

Strengths	Weaknesses
Independent governance via Fedora Council	Six-month release cycle requires periodic major upgrades
Current packages, strong hardware support	Red Hat/IBM infrastructure in the background
Btrfs default, large ecosystem	Major upgrades need more integration testing than rolling updates
Strong Flatpak and Wayland support	

Candidate B: OpenSUSE Slowroll. Slowroll occupies the space between Tumbleweed, which is fully rolling and ships new packages daily, and Leap, which follows a fixed annual release cycle. It pulls packages from Tumbleweed after a validation delay of several weeks, giving a cadence that is current enough for modern toolchains while providing a buffer against regressions. Btrfs and Snapper are more deeply integrated on OpenSUSE than anywhere else in the Linux ecosystem, which is a genuine advantage for the update strategy.

Table 5 OpenSUSE Slowroll: evaluation summary

Strengths	Weaknesses
Best Btrfs and Snapper integration in the ecosystem	SUSE owned by EQT Partners (private equity); governance risk
Rolling with validation delay; good update cadence	EQT ownership weakens European sovereignty narrative
Open Build Service is best-in-class for RPM packaging	Smaller community and hardware coverage than Fedora
European roots and presence	

The current preference is Fedora, primarily on governance grounds. The EQT ownership of SUSE is not a technical problem, but it is a liability when applying for European software independence funding through programs such as the Sovereign Tech Fund or NGI: funders will scrutinize the ownership structure and the connection to a US private equity firm sitting above a nominally European project is a difficult position to defend. Fedora's governance is cleaner for that argument. Both distributions use RPM packaging, so the Open Build Service can target either without changes to the build pipeline.

3.2 Update Strategy

Updates on most operating systems involve some degree of risk: something breaks, the user does not know why, and getting back to a working state requires either expertise or luck. This system addresses that with a layered approach that makes the worst-case outcome a single keypress rather than an afternoon in recovery mode.

The approach is a mutable filesystem with automatic rollback protection. *Mutable* means the filesystem works the way Linux filesystems always have: software can be installed freely, system files can be modified, and standard developer tools work without restrictions. This distinguishes it from the immutable distributions that have gained popularity recently, where the root filesystem is read-only and changes require going through a specific toolchain. Immutable systems have genuine safety benefits, but they break workflows that the target audience relies on, including installing language runtimes, building projects from source, and running development servers locally. The safety benefits of immutability are achievable through snapshot tooling without those costs.

Figure 3 shows the full update flow.

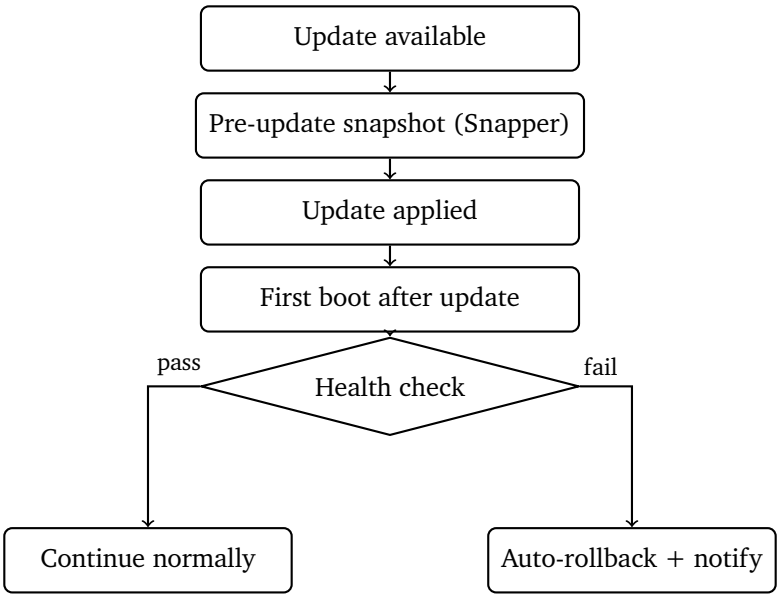

Figure 3 Update flow. A Snapper snapshot is created before every update. If the post-update health check fails, the system rolls back to the snapshot automatically.

Btrfs and Snapper. *Btrfs* is a Linux filesystem with a built-in copy-on-write mechanism: when a file changes, the old version is preserved rather than overwritten, which makes filesystem-level snapshots nearly free in terms of disk space. *Snapper* is the snapshot management tool that sits on top of Btrfs and handles creating, listing, and restoring snapshots automatically. The filesystem is divided into subvolumes, with `/` and `/home` on separate subvolumes so that a system rollback never touches personal data.

Pre-update snapshots. Before any update is applied, Snapper creates a snapshot of the current system state. This happens automatically and requires no user action; it is a precondition for the update, not an optional step.

Bootloader integration. If an update produces a system that fails to boot, the bootloader detects the failure and presents the previous snapshot as a recovery option on the next start. The user is not asked to open a terminal or enter recovery mode; they see a clear prompt with a single keypress to continue on the last working version.

Post-update health check. On the first boot after an update, a small background service verifies that critical components are functional: the display system initializes correctly, network is reachable, and no fatal errors appear in the service log. If the check fails, the system rolls back to the pre-update snapshot and notifies the user with a plain-language explanation. This catches the case where an update boots but leaves the system in a degraded state that would not be obvious until the user runs into something broken.

Snapshot retention. To prevent old snapshots from accumulating and consuming disk space, Snapper removes snapshots older than seven days automatically while always retaining the five most recent, regardless of age. Both values are adjustable in Settings.

3.3 System Infrastructure

systemd is used as-is with no replacements planned. It handles service startup ordering, dependency management, and automatic restarts for crashed services, and it is standard across both candidate distributions. `systemd-homed` manages user home directories with built-in encryption, which feeds into the account and profile system described in Section 8.

Package management at the base level uses the native manager for whichever distribution is chosen: `dnf` for Fedora, `zypper` for OpenSUSE. First-party project packages are built and distributed through the *Open Build Service* (OBS), a platform that compiles packages for multiple distributions and CPU architectures from a single set of build instructions and handles package signing and repository hosting. Store applications are distributed as *Flatpaks*, sandboxed packages that include their own dependencies and run isolated from the rest of the system, while system-level components ship as native RPMs. The Store infrastructure handles both formats transparently.

3.4 Hardware Requirements

Three hard requirements determine the minimum supported hardware. They are not adjustable without significant changes to the security and graphics stack, so they are treated as constraints rather than recommendations. In practice they exclude hardware older than roughly 2018–2019.

Table 6 Minimum hardware requirements

Requirement	Reason
Linux kernel 6.6 or newer	Required for the eBPF features used by the event monitoring system and for current security capabilities. Standard on both candidate distributions.
Vulkan-capable GPU	Required for hardware-accelerated rendering and Windows compatibility. Covered automatically on AMD and Intel; Nvidia requires the proprietary driver, with installer guidance provided.
systemd 255 or newer	Required for full `systemd-homed` functionality used by the account system.

3.5 Configuration Format

All system components and first-party applications use *TOML* as the configuration format. TOML is a human-readable format designed to be unambiguous and easy to edit manually, without the indentation pitfalls of YAML or the verbosity of JSON. Using it consistently across every component means that a user or developer who understands one configuration file understands all of them, and it means that tooling for reading, validating, and migrating configuration only needs to be written once.

Personal settings live at `~/.config/<app-id>/config.toml` and system-wide settings at `/etc/<service-name>/config.toml`. Some third-party applications expect other formats; a small helper bridges this without moving the source of truth:

```
# Export as environment variables for a legacy app
eval $(os-config-helper export \
    --toml ~/.config/myapp/config.toml --format env)

# Export as JSON for tools that require it
os-config-helper export \
    --toml ~/.config/myapp/config.toml --format json
```

The TOML file is always the authoritative source; the helper is a read-only translation layer with no business logic of its own.

4 Event System and Knowledge Graph

4.1 Overview

For the Knowledge Graph to build a picture of what is happening on the machine, something has to watch and report. The Event System is that something: it observes activity across the entire OS and feeds a continuous stream of structured events into the graph, without any visible effect on the user and without requiring applications to be rewritten for this platform.

Three sources feed into it, each covering a different slice of system activity.

Table 7 Event sources and their coverage

Source	App cooperation	What it sees
Kernel monitoring (eBPF)	None required	File opens, network connections, process lifecycle
Wayland compositor	Built-in (compositor is part of this project)	Window focus, app open/close, clipboard operations
First-party app events	Required; apps emit events directly	Rich semantic actions: document saved, search performed, tab opened

The design goal is baseline coverage for every app on the system without requiring anything from third-party developers, combined with richer semantic data where apps support it. eBPF provides the floor; structured app events raise the ceiling.

4.2 Kernel Monitoring with eBPF

eBPF (extended Berkeley Packet Filter) is a Linux kernel feature that allows small monitoring programs to run inside the kernel in a strictly sandboxed environment. These programs are verified for safety before loading and cannot crash the system, modify kernel memory, or access data outside their scope. The practical result is a read-only window into system activity at the lowest level, covering every process on the machine regardless of whether it was built for this platform.

An eBPF program attaches to a specific kernel event, such as a file being opened or a process forking, and is called each time that event fires. The output is passed to a normalizer running in user space, which translates the raw syscall data into typed, schema-validated events that the rest of the system can consume. At this layer, the system knows *that* a process opened a file but not *why* or what it means to the user. That is intentional: eBPF is the baseline, and higher-level sources fill in the semantics where they are available.

The eBPF implementation uses `aya`, a Rust-native eBPF library that requires no C code and integrates cleanly with the rest of the codebase.

Noise filtering. eBPF can generate thousands of events per second under normal load, most of which carry no useful information for the Knowledge Graph. Two filters in the normalizer reduce this before events reach the bus. Identical operations on the same file by the same process within 100 milliseconds are collapsed into a single event with an updated timestamp. Paths under `/proc`, `/sys`, `/dev`, `/tmp`, and shared libraries under `/usr/lib` are discarded entirely, since they have no semantic value in a graph of user activity.

4.3 Event Bus

The Event Bus is a background daemon that receives events from all sources, validates them against the event schema, and forwards them to registered consumers. Its only job is to move and validate data; it does not interpret or store anything. Figure 4 shows the full pipeline.

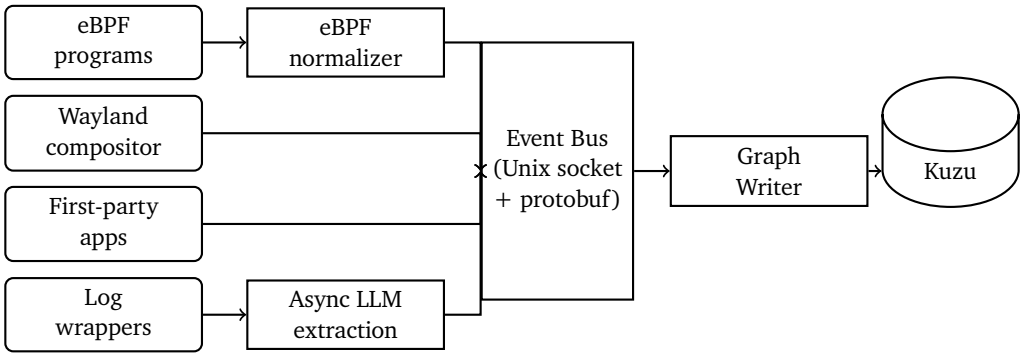

Figure 4 Event pipeline. Raw kernel events pass through a normalizer before reaching the bus. Log output from third-party applications is processed asynchronously by an LLM that extracts structured entities. All sources converge on the Event Bus, which validates and forwards to the Graph Writer Daemon.

Event schema. Every event, regardless of source, shares a common envelope that carries a time-sortable UUID v7 identifier, a type label, a microsecond timestamp, a source identifier, the originating process ID, and a session reference. The type-specific data travels as a protobuf payload inside the envelope.

```
message Event {
  string id = 1; // UUID v7 (time-sortable)
  string type = 2; // "file.opened", "window.focused", etc.
  int64 timestamp = 3; // microseconds since epoch
  string source = 4; // "ebpf", "wayland", "app:<id>"
  uint32 pid = 5; // originating process
  string session_id = 6; // user session reference
  bytes payload = 7; // type-specific protobuf message
}
```

Listing 1 Common event envelope (protobuf)

A file.opened payload carries the file path, the application name, and the operation flags. A window.focused payload carries the app ID and window title. App events use a generic structure with a category, an action, a subject, and an optional metadata map, which allows any first-party application to emit meaningful events without requiring a dedicated payload type per application.

Clipboard content is never logged. The bus records that a clipboard operation occurred and its MIME type, not the content itself.

D-Bus coexistence. The custom Event Bus handles communication between project-internal components only. D-Bus remains in place for everything it already handles: desktop notifications, NetworkManager, Bluetooth, session management via systemd-logind, sandboxed app portals, and standard third-party applications. The two systems run in parallel with no overlap; nothing that communicates over D-Bus today is changed.

4.4 Backpressure and Batching

Kuzu is an embedded graph database optimized for analytical queries over large graphs, not for high-frequency single-row writes. Writing each eBPF event individually as it arrives would saturate the write path quickly. Instead, the Graph Writer Daemon maintains a ring buffer and writes to Kuzu in batches.

> **Phase 0 validation required.** The batching architecture below is designed to make Kuzu viable under sustained eBPF load. This must be confirmed with a benchmark before the architecture is finalized: Event Bus plus Graph Writer under simulated load targeting 5,000 events per second, with Kuzu as the write target. If Kuzu cannot sustain this load even with batching, the fallback is a two-layer architecture with SQLite handling writes and Kuzu receiving promoted data periodically for queries. This is a Go/No-Go decision at Phase 0.

The ring buffer holds 10,000 event slots. A batch write is triggered by whichever threshold is reached first: 500 milliseconds have elapsed, or 1,000 events have accumulated. Both values are configurable. When the buffer is full, the system works through three tiers before discarding anything. First, it checks whether an identical event (same type, source, and subject) already exists in the buffer and, if so, updates its timestamp rather than inserting a duplicate. If no duplicate exists, it drops the lowest-value event currently in the buffer to make room, prioritizing raw eBPF read and write events with no app-level context. Only if neither approach frees a slot is the incoming event discarded, which should not occur under normal operating conditions. Drop rates per tier are exposed as metrics and monitored by the Anomaly Detector.

4.5 Knowledge Graph

Why a graph. A relational database stores relationships as foreign keys across tables. A graph database stores them as first-class edges. The difference is felt most when querying across multiple entity types simultaneously: finding which files a user worked on last Tuesday, which applications touched them, and whether any of those applications are still running requires three joins and a subquery in SQL and a single traversal in a graph. For an operating system where everything is connected (processes open files, files belong to projects, projects span sessions, sessions overlap with other applications), a graph models the domain naturally.

Kuzu. The Knowledge Graph is implemented with *Kuzu*, an embedded graph database written in C++ with Rust bindings, released under the MIT license. Embedded means

it runs inside the Graph Daemon process with no separate server, no network socket, and no additional service to manage. Kuzu uses *Cypher* as its query language, a declarative syntax that reads close to natural language and is already familiar to anyone who has worked with Neo4j.

```
MATCH (f:File)-[:OPENED_BY]->(a:App)
WHERE f.last_modified > datetime('2026-03-01')
RETURN f.path, a.name
ORDER BY f.last_modified DESC
```

Listing 2 Example Cypher query: files accessed last Tuesday

Schema. Figure 5 shows the core node types and their relationships. The schema is intentionally minimal at this stage; it will grow as the system matures. File, App, Session, Project, Event, and UserAction are the primary entities. Every raw occurrence from the event pipeline lands as an Event node first; higher-level UserAction nodes are derived from clusters of events, potentially with LLM assistance, and represent meaningful activity rather than individual syscalls.

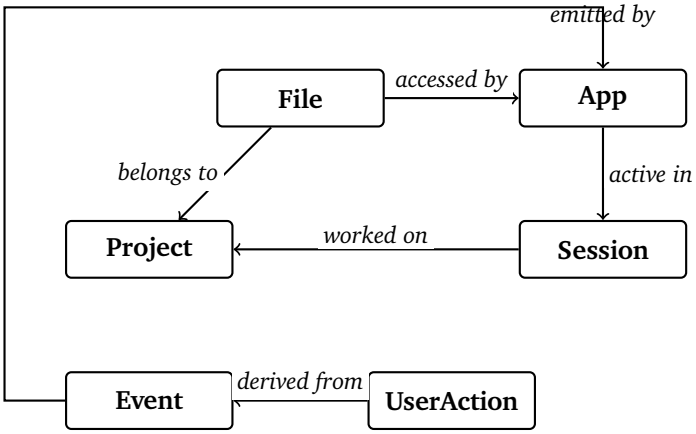

Figure 5 Knowledge Graph core schema. Nodes represent entities; edges represent relationships. Event nodes are the lowest-level entries, created directly from the event pipeline. UserAction nodes are derived abstractions built from clusters of raw events.

4.6 Retention Policy

The graph accumulates data indefinitely if left unchecked, which creates two problems: storage grows, and the privacy surface area expands. The retention policy defines how long data lives, with user control at every tier.

Summarization. Hard deletion of aged nodes loses information abruptly. Instead, nodes past the retention window are compacted into summary nodes that retain aggregate statistics (access count, primary application, active period) while discarding the individual event edges. The node still exists in the graph and AI queries still find

Table 8 Knowledge Graph retention tiers

Tier	Default	Content
Raw eBPF events	30 days	High-volume, low-semantic-value syscall events. Already incorporated into higher-level nodes after ingestion; the raw records are largely redundant beyond the anomaly detection window.
Semantic nodes	12 months rolling	File, App, Session, and NetworkConnection nodes representing actual activity. This is where AI query value lives. Nodes past the window are summarized rather than deleted.
Pinned nodes	Permanent	Nodes explicitly marked by the user, active Project nodes, and nodes recently referenced in AI queries. Only manual deletion removes these.

it; the granularity is reduced but the semantic content is preserved. Summarization is triggered by access frequency rather than age: a node that has not been queried in six months is a candidate regardless of when it was created, while a node from two years ago that is queried regularly remains granular. A background daemon identifies candidates nightly and processes them during idle periods.

User controls. Settings exposes the retention window per tier with documented minimums, a manual summarize-now option for specific time ranges, node and subgraph deletion, full graph export for data portability, and storage usage broken down by tier and time range. In managed deployments, administrators can set a minimum retention for compliance-relevant audit events in a separate audit log, but they cannot shorten the user's configured retention window for personal graph data.

5 AI Layer

5.1 Overview

The AI layer sits on top of the Knowledge Graph and provides both the user and the system with access to an AI model that has structured context about what is happening on the machine. It is not a chatbot grafted onto an existing OS; it is an interface that can query the graph, invoke application capabilities through a standardized protocol, and optionally act autonomously when the user explicitly enables that.

The AI layer is not the primary reason this system exists. The desktop, the permission system, the profile isolation, and the store all stand on their own. AI is an enhancement

that becomes more useful as the Knowledge Graph fills in over time, and users who want none of it can disable it completely without losing any other functionality.

Three principles govern the design of this layer.

Query-first. The AI responds when asked. Nothing happens in the background without an explicit trigger. Autonomous features exist but are separate opt-ins, not defaults that the user has to find and disable.

Provider-agnostic. The system does not care whether the model runs locally or in the cloud. Every provider implements the same interface; the rest of the codebase never speaks to a specific backend. The user decides what runs where, including routing sensitive requests to a local model only.

Opt-out at any point. The entire layer can be disabled in Settings under AI. When disabled, no local model runs, no graph queries originate from the AI layer, Spotlight falls back to standard search, and no MCP servers are active. The setting is not hidden five menus deep.

5.2 Cold-Start Behavior

The Knowledge Graph is empty on a fresh install. An AI feature that promises contextual awareness but has nothing in the graph yet is worse than no feature at all: it creates an expectation and immediately fails to meet it. The system handles this with straightforward communication rather than by faking usefulness.

During onboarding, AI features are presented with an honest explanation of what to expect: the assistant learns context gradually as the system is used, starts with nothing, and becomes more useful over days and weeks. There are no demo interactions, no placeholder context, and no fabricated examples of what it will eventually be able to do.

At onboarding the user can optionally seed the graph with existing data from their current system: browser history, calendar events, and documents. This is presented as a trade-off, not a recommendation. Each source can be imported independently, so someone who is comfortable sharing their calendar but not their browsing history can import only the former. Importing gives the AI immediately useful context without a multi-week waiting period.

The AI assistant does not surface proactively until a threshold is met: seven days of usage or a completed manual import. Before that threshold the feature is accessible but does not advertise itself. The non-AI parts of the system are functional and complete from the first boot, so there is no gap in usefulness during this period.

5.3 Provider Abstraction

Different AI providers have different APIs, different latency profiles, and significantly different privacy implications. Sending the contents of a private file to a cloud API

is appropriate in some contexts and completely unacceptable in others. The system needs to handle both without the rest of the codebase caring which provider is active.

The solution is a Rust trait that every AI backend implements. The AI layer constructs a `CompletionRequest`, passes it to whichever provider is configured, and receives a `CompletionResponse`. No provider-specific code is visible outside the provider module.

```rust
#[async_trait]
pub trait AiProvider: Send + Sync {
    async fn complete(
        &self,
        request: CompletionRequest,
    ) -> Result<CompletionResponse>;
    async fn available(&self) -> bool;
    fn name(&self) -> &str;
}
```

Listing 3 AI provider trait

Figure 6 shows how the AI layer relates to the available backends.

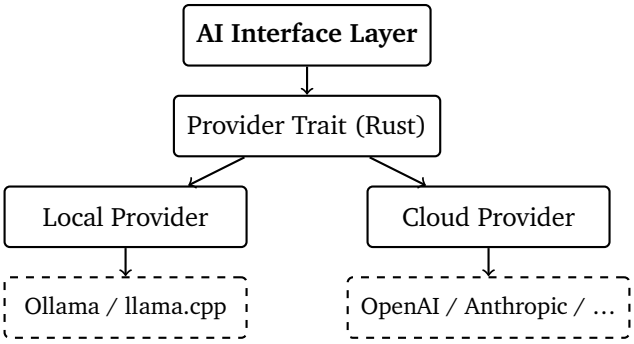

Figure 6 Provider abstraction. The AI layer speaks only to the trait; the concrete backend is a configuration choice. Local and cloud providers are interchangeable from the perspective of anything above the trait boundary.

Multiple providers can be active simultaneously under a routing policy configured by the user: a local model as the default, with fallback to cloud if the local model is unavailable or too slow; specific task types such as code always routed to a designated provider; and sensitive contexts, such as requests involving personal file contents, explicitly prevented from leaving the machine.

5.4 MCP Integration

MCP (Model Context Protocol) is an open protocol that standardizes how AI models interact with external tools and data sources. Before MCP, every AI integration was bespoke: one application called the OpenAI API directly, another used its own tool-calling format, and combining them required custom glue code on both ends. MCP standardizes this at the protocol level so that any MCP-compatible AI client can communicate with any MCP server without application-specific integration work.

An MCP server is a small process that exposes a set of tools the AI can call and resources the AI can read. In this system, first-party applications expose MCP servers

as part of their standard implementation: the file manager exposes list_directory, open_file, and move_file; the terminal exposes run_command and get_output; the Knowledge Graph exposes a query interface. The AI can coordinate across multiple applications in a single interaction by calling their respective servers in sequence. Figure 7 shows the topology.

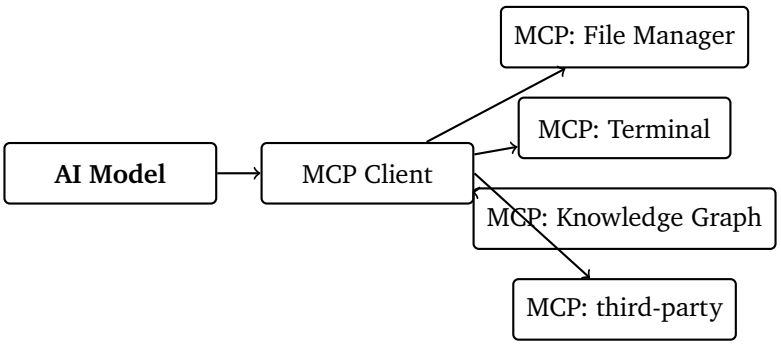

Figure 7 MCP topology. The AI model communicates through a single MCP client that dispatches to individual application servers. Third-party applications from the Store can ship their own MCP servers, subject to the permission model described in Section 5.6.

CLI tools do not need MCP servers. If a tool has a –help flag and produces sensible output, the AI can invoke it as a subprocess and read stdout. MCP is reserved for GUI applications and stateful interfaces where a subprocess call is insufficient.

5.5 Interaction Model

Phase 1: Query-based (default). The user asks a question, the AI responds. No background activity occurs without an explicit trigger. The AI has access to the Knowledge Graph and whatever MCP servers are permitted under the current configuration. Typical interactions include querying session history, finding files related to a project, correlating system events with observed behaviour, and composing multi-step actions across applications.

When a question is asked, the AI layer translates the natural language input into a Cypher query, executes it against the Knowledge Graph via the Graph Daemon, and passes the structured results to the model as context. Relative time expressions are resolved to absolute UTC timestamps before the generation step, so the model always operates on concrete values rather than ambiguous phrases. The Cypher generation and the result formatting are two separate model calls, since combining them causes the model to bias the query toward a desired-sounding answer rather than accurately querying the schema.

The query validator sits between generation and execution: it checks that the generated Cypher uses only known node types and relations, contains no write operations, and includes a LIMIT clause. If validation fails, the error is fed back for a retry; after two failed attempts the user is told the question could not be answered, with no silent fallback.

Phase 2: Autonomous agent (explicit opt-in). A background agent that observes the event stream and acts proactively is available as an opt-in capability. It is not enabled by default and each autonomous behaviour is a separate toggle: suggesting related files when a document is opened is independent of automatically tagging new files by project. The agent uses the same provider abstraction and MCP layer as the query interface; the difference is that it initiates actions rather than responding to them.

5.6 MCP Permission Model

MCP servers are the interface through which the AI acts on applications. The permission model distinguishes between reading information, which the AI can do by default within the user's configured AI permission level, and modifying state, which requires explicit per-session authorization.

Table 9 Read-only MCP servers (permitted by default)

Server	Exposes
File Manager	Directory listings, file metadata
Calendar	Upcoming and past events
Browser	History and open tabs (no passwords, no form data)
System Monitor	Process list and resource usage
Notes	Note titles and content

Reading to answer a question is what the user expects when they ask the AI something. Answering "what meetings do I have today?" without a confirmation dialog is the correct behaviour. Reading does not change state, and the AI permission level configured in Settings still gates what data each server exposes regardless of this default.

Table 10 Write and action MCP servers (per-session authorization required)

Server	Can do
Terminal	Execute shell commands
File Manager	Create, move, rename, delete files
Calendar	Create and modify events
Email	Compose and send messages
Store	Install and remove packages

Authorization is granted once per session and covers the scope the user specifies: approving file management does not imply approval for email. It expires at session end and is not persisted. Within an authorized scope, a separate hardcoded list of actions always requires explicit confirmation regardless of session permissions, because their consequences are irreversible or high-impact: permanent file deletion, sending any external message, installing or removing system packages, modifying system configuration outside ~/.config, any network request to a host outside the application's

declared permissions, and running commands with elevated privileges. This list is not configurable.

Third-party MCP servers shipped by Store applications are always treated as action-requiring by default. A community server that claims to be read-only receives the same treatment until it carries a verified security audit covering its MCP implementation.

6 Desktop Environment and Design Language

The desktop environment is the most visible part of the system and the primary differentiator for users switching from other platforms. The goal is a fast, coherent desktop that does not feel like a collection of independent projects assembled by different teams in different decades. The stack reflects this: performance-critical and system-close code is Rust, everything visual is TypeScript with Tailwind CSS, and the two halves communicate over well-defined interfaces that neither side reaches past.

6.1 Architecture and IPC

Three communication channels connect the compositor core to the shell and its frontend, each carrying a different class of data. Figure 8 shows the structure.

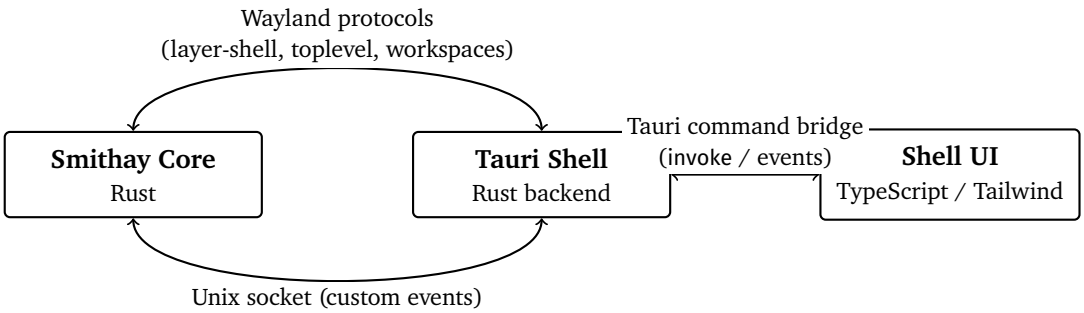

Figure 8 Shell IPC architecture. Wayland protocols carry window management data (open windows, focus, workspace state). The Unix socket carries system-specific events with no Wayland equivalent: theme updates, module lifecycle events, graph notifications. The Tauri command bridge connects the Rust backend to the TypeScript frontend within the shell process.

Wayland protocol extensions are the correct channel for anything the Wayland ecosystem has standardized. The Unix socket exists for everything that does not have a protocol equivalent and where a custom solution is both simpler and more direct. Messages on the socket use JSON during development, with migration to protobuf planned once the schema stabilizes.

Within the Tauri shell, the Rust backend and TypeScript frontend communicate through Tauri's typed command system: the frontend calls Rust via `invoke()`, Rust pushes events to the frontend via Tauri's event emitter. No ad-hoc serialization; the bridge handles it.

6.2 Compositor and Window Manager

The Wayland compositor is built on *Smithay*, a Rust framework that provides the low-level plumbing for compositor development: Wayland protocol handling, DRM/KMS interfaces for direct display access, input processing via libinput, and buffer management. Compositor logic such as window placement, focus rules, and workspace behaviour is built on top of Smithay rather than dictated by it. Building a compositor without a framework is an undertaking that would dominate development for years; Smithay provides the foundation without constraining the design. The COSMIC desktop compositor from System76 (cosmic-comp) is a useful reference for a Rust-first desktop project with comparable goals.

Window management supports both floating and tiling layouts, switchable per workspace. There is no forced paradigm: users who prefer floating windows on one workspace and a tiling layout on another can configure that independently per workspace.

The compositor implements the Wayland protocol extensions listed in Table 11.

Table 11 Implemented Wayland protocol extensions

Protocol	Purpose
wlr-layer-shell	Positions the taskbar and panels at screen edges, above application windows
wlr-foreign-toplevel-management	Exposes open windows (title, app ID, state) to the taskbar
wlr-workspace-management	Workspace state and control for the taskbar
xdg-shell	Standard protocol for normal application windows
xdg-output	Monitor metadata for multi-monitor configurations
xdg-desktop-portal	Screensharing, file picker, and OS-level dialogs for sandboxed apps

XWayland. X11 compatibility is provided via XWayland, an X11 server that runs as a Wayland client, making legacy X11 applications transparent to the user. It starts on demand when the first X11 application launches and shuts down when none are running. Users who do not need X11 compatibility can disable it in Settings to reduce the attack surface.

XWayland has structural limitations that cannot be solved at the compositor level and are documented rather than hidden. Copy-paste between X11 and Wayland applications requires a clipboard synchronization bridge. Drag and drop across protocol boundaries is unreliable. X11 applications cannot use xdg-desktop-portal for screensharing and see only the XWayland surface. Most significantly, X11 has no inter-application isolation: one X11 application can read the input events of any other

X11 application, which is a keylogger-class capability. This is recorded in the audit log whenever X11 applications are active.

6.3 Shell and Application Framework

The shell, comprising the taskbar, app launcher, notification center, quick settings panel, and lock screen, is built with Tauri using TypeScript and Tailwind CSS for the interface layer. Tauri renders the UI in a WebKitGTK WebView, which gives the shell access to the full set of modern web rendering capabilities without the memory cost of an Electron-style approach. The shell is a Wayland client that uses `wlr-layer-shell` to anchor its surfaces to screen edges.

ui-kit. A shared TypeScript package provides the visual foundation for every first-party application. It is based on *shadcn/ui*, a set of Tailwind-based components that are copied into the project and fully owned rather than consumed as an npm dependency. The package is organized in three layers: design tokens as CSS custom properties that define the visual language and propagate everywhere when changed; base components such as Button, Input, Dialog, and Card built on shadcn/ui and styled with those tokens; and system-specific components such as `WindowChrome`, `AppShell`, and `NotificationToast` that implement OS-level interaction patterns. Every first-party application imports from ui-kit and receives visual consistency without duplicating any of this infrastructure.

os-sdk. A shared Rust crate provides system integration to every first-party application. It bundles a Graph Daemon client, an Event Bus client, MCP server boilerplate, and the TOML-based config system. A new first-party application starts with Tauri, ui-kit, and os-sdk and has graph access, event emission, an MCP server, config handling, and a consistent UI without writing any infrastructure code. The standard application layout reflects this:

```
app-files/
  src-tauri/
    Cargo.toml # includes os-sdk
    src/
      main.rs # Tauri setup
      commands.rs # Tauri command handlers
      mcp.rs # this app's MCP server
  src/
    main.tsx
    components/ # app-specific components (uses ui-kit)
    pages/
```

Listing 4 Standard first-party app structure

6.4 Spotlight

Spotlight is the system-wide launcher and command palette: the primary keyboard-driven entry point to the entire desktop, triggered by the Super key by default and

accessible from any context as a centered overlay. It handles app launching, file and web search, inline calculation, unit conversion, timezone lookup, and contextual actions derived from the current selection or active application. It is not a file manager; deep folder navigation and bulk operations belong in the File Manager. Spotlight surfaces the result the user wants immediately or hands off to the right application.

When the AI layer is active, Spotlight gains an AI mode toggled by Tab. Natural language queries are interpreted by the AI layer, resolved against the Knowledge Graph and available MCP actions, and returned as structured results in the same interface. Asking "show me what I worked on this week" is a Spotlight query, not a file search.

Third-party modules can register as `spotlight.provider` to add result types or as `spotlight.action` to add commands, as described in Chapter 7.4.

6.5 Multi-Monitor and HiDPI

Wayland treats multi-monitor and HiDPI as first-class concepts: each output carries its own scale factor, every Wayland-native application queries its output properties and renders at the correct scale automatically, and WebKitGTK respects the Wayland scale factor transparently. This means Tailwind layouts render identically on 4K and 1080p without extra work in the application.

The taskbar runs as a separate `layer-shell` surface per monitor, each correctly sized and scaled for its output. Monitor configuration, covering resolution, refresh rate, scale factor, position, and rotation per display, is stored in `~/.config/display/config.toml` and applied at compositor startup. When a new monitor is connected, the compositor checks for a saved configuration and applies it immediately if found; for an unknown monitor it applies sensible defaults and surfaces a notification offering to open the display settings.

Fractional scaling at 1.5x (common on 1440p displays) uses the `wp-fractional-scale-v1` Wayland protocol. Applications that implement the protocol render crisply at fractional scales; those that do not are rendered at the nearest integer scale, which results in slightly soft output but is acceptable and documented. XWayland has no per-monitor DPI concept and uses a single global DPI matched to the primary monitor; X11 applications do not re-scale when moved between monitors, which is a structural X11 limitation with no compositor-level fix.

6.6 Design Language and Theming

The specific visual design language, exact color palettes, and motion design are finalized in a separate design phase. This section documents the technical theming architecture: how themes are structured, how they propagate, and how community themes work.

Token system. All themes are defined in TOML and compiled to platform-specific outputs by the build system. The TOML file is the single source of truth; no theme value exists anywhere else. From one `tokens.toml`, the build generates CSS custom

properties for the Tauri shell and all first-party applications, a GTK4 CSS theme, a Wine .msstyles file for Windows compatibility, and base dark/light palettes for GTK3 and Qt. Changing the accent color in the TOML propagates to every layer on the next build.

Built-in themes. Four themes ship with the system. Dark and Light are standard high-contrast variants. High Contrast targets WCAG AAA at a minimum 7:1 contrast ratio, with no transparency effects, no gradients, and no information conveyed through color alone; it is generated from the same token system as the others and has both a dark and light variant. Panda is the default and the most distinctive: a deliberate split-surface design where different UI regions are intentionally dark or light based on their function, creating a high-contrast frame around bright content. The exact palette values for all four themes are determined in the design phase; the token structure that generates them is fixed.

GTK and Qt coverage. GTK4 receives a full custom theme generated from tokens, covering the standard set of interactive components with recognizably the same design language as the first-party applications, though not pixel-perfect parity. GTK3 and Qt receive fixed base dark and light themes only; the split-surface Panda design requires component-level CSS control that neither framework provides cleanly, so they fall back to whichever base is closer to the active theme.

Store themes. Community themes are distributed through the Store. A theme package declares which layers it covers in its manifest and can inherit from any built-in theme, overriding only the values it changes. The Store displays coverage clearly before installation so users know which layers remain at the base theme. Two authoring formats are supported: token overrides for authors who want to change colors and radius values without touching CSS, and full compiled CSS for themes that need structural changes beyond what the token system exposes. Both formats can coexist in one package.

Live switching. Figure 9 shows the theme inheritance chain. When the user applies a theme, the Settings daemon loads the package, merges its token overrides with the base theme, broadcasts new CSS custom properties to all running Tauri processes (which update instantly without a reload), regenerates and applies the GTK4 theme if the package covers it, and updates Wine prefix themes if applicable. Token-based switches complete in under a second; GTK4 regeneration may take one to two seconds on slower hardware.

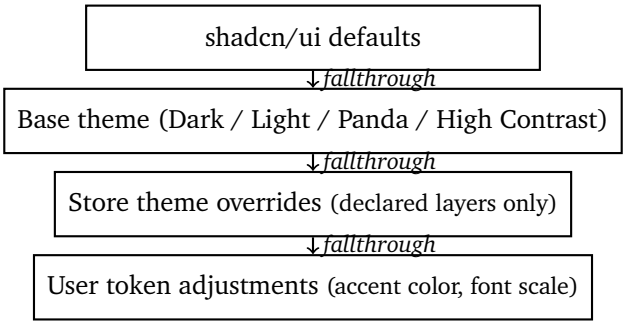

Figure 9 Theme inheritance chain. Each layer overrides only what it declares; everything else falls through to the layer below. A Store theme that covers only the shell and GTK4 leaves Wine and Qt at the base theme.

7 Apps, Modules, and the Store

7.1 App Ecosystem

The application layer divides cleanly into two categories: first-party applications built as part of this project, and the broader ecosystem of third-party Linux software. Both operate under the same permission system and benefit from the same infrastructure, but they differ in how deeply they integrate with the platform.

First-party applications. All core system applications are built with the standard stack: Tauri, ui-kit, and os-sdk. Each one follows the shared design language, implements an MCP server for AI integration, emits structured events to the Event Bus, and uses the shared TOML configuration system. Table 12 lists the planned core applications.

Table 12 First-party system applications

Application	Notes
File Manager	Graph-aware features: recent files by project, related files sidebar
Terminal	MCP server exposing command history and session output
Settings	System-wide configuration: theming, permissions, AI level, accounts
Store	App and module discovery, installation, and updates
Text Editor	Basic editor primarily for configuration files
System Monitor	Process list, resource usage, and Anomaly Detector alerts
Wine Manager	Prefix management, Winetricks access, per-prefix version selection

Third-party integration layers. Third-party applications are not rewritten. Integration happens in four layers of increasing depth, each building on the one below.

At the baseline, every application on the system receives passive tracking via eBPF regardless of whether it does anything to support this platform: file access, network

connections, and process lifecycle are recorded automatically. No changes to the application are required.

For applications that produce useful log output, a wrapper captures stdout and stderr and passes them through asynchronous LLM extraction to produce graph entities. These wrappers are shipped alongside packages in the package manager where practical, and the extraction runs in the background without blocking anything.

For popular applications, dedicated MCP server wrappers expose structured interfaces to the AI layer. These are maintained as separate packages and where possible contributed back to the community as independently useful tools.

Finally, applications that explicitly support the platform APIs emit structured typed events directly to the Event Bus and implement native MCP servers. This is the long-term goal for major open-source applications but is not a precondition for any of the layers above it.

Shared standards. Applications that choose to integrate with the platform work with a common set of conventions: TOML configuration files under ~/.config/<app-id>/, CSS custom properties from the token system for applications that render via WebView, the standard notification daemon via the Freedesktop D-Bus interface or the Event Bus for richer notifications, and a published MCP server interface specification.

7.2 Notification System

The Notification Daemon is the single entry point for all notifications from all sources. Figure 10 shows the pipeline.

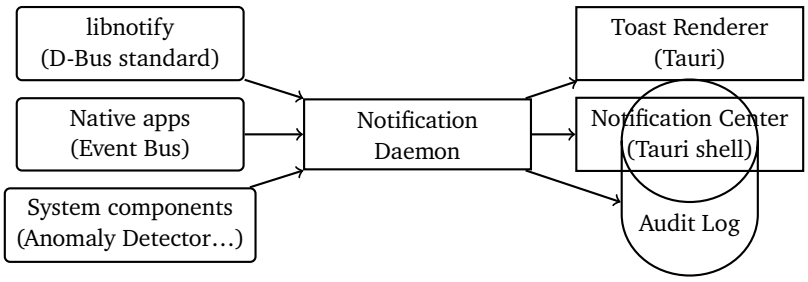

Figure 10 Notification pipeline. All sources converge on the Notification Daemon, which normalizes, groups, and applies priority logic before forwarding to the toast renderer and notification center. Every notification is logged in the Audit Log by sender and timestamp, never by content.

The daemon receives from two sources: the standard D-Bus interface at org.freedesktop.Notifications, which means every third-party application using libnotify or notify-send works without changes; and the Event Bus, through which native system components send richer notifications with priority metadata and graph context. Both are normalized to a common internal schema before any further processing.

Critical priority is reserved for system-level alerts only: Anomaly Detector warnings, component compatibility failures, and security events. Applications cannot self-assign Critical. Notifications from the same application are grouped in the Notification Center;

Table 13 Notification priority levels

Level	Behavior	Do Not Disturb
Critical	Persistent toast, no auto-close	Always shown, never silenced
High	Toast auto-closes after approximately 8 seconds	Delayed, not blocked
Normal	Toast auto-closes after approximately 4 seconds	Silenced
Low	No toast; goes silently to Notification Center only	Silenced

as a toast, only the most recent notification from an application is shown, with a counter if a previous one from the same application is still visible. Do Not Disturb has three modes (off, on, and scheduled by time of day) and is togglable from the taskbar quick panel.

7.3 Permission System

Every application on the system, whether a first-party app, a Flatpak, or an RPM package, operates within a permission profile that defines what it can access. The system is unified: one file format, one enforcement layer, one place for the user to review and adjust everything.

Permission file format. Each application has a TOML permission file at ~/.config/permissions/<app-id>.toml that serves as the sole source of truth for what that application is allowed to do. The file is human-readable and user-editable; the Settings app provides a UI for those who prefer not to edit it directly. Permissions are enforced via Linux namespaces, AppArmor, and seccomp, described in Section 8.4.

```toml
# ~/.config/permissions/org.mozilla.firefox.toml

[filesystem]
allow = ["~/Downloads", "~/Documents", "~/.mozilla"]

[network]
allow = ["full"]

[devices]
allow = ["gpu"]

[graph]
# no entry = no access, never prompted
```

Listing 5 Example permission file

Graph access is never inferred or learned. An application with no [graph] section has no graph access, with no exceptions and no automatic prompting. Graph permissions must be explicitly declared.

Store-supplied profiles. The Store ships a curated permission profile alongside known applications. Common software such as Firefox, LibreOffice, and VLC has community-maintained profiles that are installed automatically and shown to the user at install time for review. Community members can submit profiles for applications not yet covered; submitted profiles go through Store review before publication, since an overly permissive profile is a meaningful risk.

Flatpak adapter. Flatpak applications carry their own permission declarations. When a Flatpak is installed, the system automatically generates a native permission file by mapping Flatpak permissions to the native format: `filesystem=home` becomes `filesystem.allow = ["~/"]`, and so on. The generated file is shown to the user at install time and can be tightened before confirming. The Flatpak manifest is the floor; the system never grants more than what the Flatpak declared.

First-run learning mode. If no Store profile exists and the application is not a Flatpak, the first launch runs in learning mode: all accesses are permitted and logged, nothing is blocked. After the session ends, the user receives a summary of what the application accessed, with options to review and confirm, edit first, or start with all permissions blocked. From the second launch onward, the confirmed profile is enforced. Learning mode applies to filesystem, network, and device access only; graph access is always denied until explicitly granted and cannot be learned.

Runtime behavior. When an application attempts an access outside its profile, the access is denied and the user receives a notification offering to extend the profile. If the user allows, the profile is updated and the application retries the action; if the user denies, the block stands. Repeated denials for the same path are collapsed: after three denials for the same access, the system asks once whether to always block it rather than continuing to prompt.

7.4 Module System

The system is extensible at multiple layers through a unified module mechanism. All module types use the same installation channel (the Store) and the same manifest format, but they operate as independent subsystems with separate APIs. A single Store package can bundle multiple module types together.

Module categories. Table 14 lists the available extension points. Theme modules are a special case: they consist of CSS and token overrides only, contain no executable code, and require no sandbox. All other categories run sandboxed with declared permissions.

Manifest format. Every module package has a single manifest that declares what it provides, its dependencies, conflicts, and permissions. The `[permissions]` block uses the same schema as application permission files; at install time the Module Runtime

Table 14 Module categories and extension points

Category	Extension point	Example
theme	Design language layer	Community color schemes
spotlight.provider	Spotlight search results	Web search, stock prices
spotlight.action	Spotlight command actions	Create task, send message
shell.topbar	Topbar element slot	Custom status widget
shell.widget	Desktop widget area	Clock, weather, system stats
mcp.server	AI MCP integration	External service connector
keybinding.profile	Global input manager	Alternative layout profiles

generates the permission file from the manifest and the user can inspect and adjust it in Settings like any other application.

```
[package]
id = "com.example.ai-spotlight"
name = "AI␣Spotlight␣Provider"
version = "1.0.0"

[provides]
spotlight.provider = true

[dependencies]
shell = ">=1.2"
modules = ["com.example.ai-core"]

[permissions]
network.allow = ["api.openai.com"]
graph.allow = ["read"]
```

Listing 6 Example module manifest

Modules cannot declare permissions beyond what the manifest specifies, and manifests are reviewed during Store submission. Unlike applications, modules cannot expand their permissions at runtime through a learning mode; their profile is set at install time and any undeclared access returns `EACCES`.

Module Runtime. A dedicated Module Runtime daemon manages lifecycle, sandboxing, and crash recovery for all active modules. Each module runs in its own isolated WebView context, separate from the shell's WebView. A module crash does not affect the shell or other modules.

The runtime uses exponential backoff per module instance for crash recovery, shown in Table 15. The 60-second window resets on a clean run, so a module that runs for several minutes and then crashes once gets an immediate restart.

When a module is marked failed, Settings shows a subtle error indicator with the last crash log. The user can trigger a manual retry at any time. There is no notification spam.

Table 15 Module crash recovery policy

Crash count (within 60 s)	Behavior
1st	Immediate restart
2nd	Restart after 5 seconds
3rd	Restart after 30 seconds
4th or more	Marked as failed; no further automatic retries

Module updates are downloaded in the background and applied at different points depending on category: theme updates apply immediately as a token and CSS swap; Spotlight modules update on the next Spotlight open; shell widgets and topbar modules update on the next session start by default, with a manual "Restart now" option available in Settings; MCP server modules update on the next session start to avoid interrupting active AI interactions.

Shell API. Modules interact with the system through a restricted TypeScript SDK. No direct access to the Smithay compositor or Graph Daemon is available; what is not in the SDK is not accessible.

```
import { shell } from "@os/module-sdk"

// Window management
const windows = await shell.getWindows()
const active = await shell.getActiveWindow()

// Graph (requires graph = "read" in manifest)
const recent = await shell.graph.query(
  "MATCH␣(f:File)␣RETURN␣f␣LIMIT␣5"
)

// Spotlight provider registration
shell.spotlight.register({
  id: "com.example.ai-spotlight",
  placeholder: "Ask␣AI...",
  onQuery: async (q) => [ /* result objects */ ]
})
```

Listing 7 Module SDK surface (TypeScript)

Global Input Manager. Keybindings and gestures are configured in a single place in Settings, not per application or per module. Modules register named actions with default bindings; the user overrides them centrally. The module never receives the binding string directly; the Input Manager calls the registered action handler when the binding fires. This means a user can rebind everything from one place and no module can silently capture input outside its declared actions.

Developer experience. A CLI scaffolds a complete module project with one command: `npx create-os-module <name>`, producing a Vite, TypeScript, Tailwind, and module-sdk

setup with a manifest template. In dev mode, the module runs unsigned in a dedicated dev sandbox, all declared permissions are granted without Store review, WebView DevTools are accessible, and hot-reload via Vite is active. Dev mode is only available on developer builds; production builds reject unsigned modules.

For distributing a finished module outside the Store, a developer generates a self-signed key pair with the CLI, signs the module, and distributes it as an `.osmod` file. Installation uses TOFU: the first install prompts the user to trust the key fingerprint; subsequent updates from the same key are silent. Sideloaded modules receive no expanded permissions and are marked "Not from Store" in Settings.

7.5 Store

The Store is the primary distribution channel for applications, themes, modules, and profile packages. It is designed to be financially sustainable for the project without becoming a gatekeeper for the ecosystem.

Publishing. Publishing an application to the Store is free and remains free indefinitely. There is no annual developer fee. Every upload runs automated technical checks: valid permission declarations, correct package signing, sandbox policy compatibility, and resolvable declared dependencies. These checks are automated; no manual review is required for packages that pass them. A failed check returns a specific error. A manual Security Review is available optionally and is the basis for the Security Audit Badge described below.

Revenue model. Supported pricing models include free, one-time purchase, subscription, in-app purchases, pay-what-you-want, and freemium. Developers can also direct users to external payment links; the Store payment infrastructure is a convenience, not a monopoly.

Commission is calculated on total annual revenue across all applications in a developer account, using the tiered structure in Table 16. When a threshold is crossed mid-year, the new rate applies to all subsequent transactions in that year, not retroactively.

Table 16 Store commission tiers

Annual revenue (account total)	Commission
Up to 10,000€	0%
10,000€ to 50,000€	5%
50,000€ to 500,000€	15%
500,000€ to 5,000,000€	20%
Over 5,000,000€	25%

User-facing features. Purchases are tied to the user account, not the device. Trials are declared in the app manifest and enforced by the Store; no credit card is required to

start a trial, and when it ends the user receives a single notification with a purchase option and no automatic charges. Family Sharing is opt-in per application: developers can allow up to six to eight family members to use an application under one purchase. Every application's Store page shows a full price history including every price change with its date; this is automatic and cannot be disabled by the developer. All active subscriptions across all applications are visible and cancellable in one place in Settings.

Developer dashboard. Developers see install counts, active user numbers, revenue and payout history, ratings and written reviews, crash reports from users who opted into crash reporting at the system level, and version adoption numbers. All data is aggregated and anonymized; the developer sees counts, never identities. No third-party analytics are required or supported.

Security Audit Badge. Applications can display a Security Audit badge on their Store page by uploading an independent security audit report. The Store verifies that the report originates from the stated auditing organization and links it from the application page. Users can read the full report and judge the auditing organization's credibility themselves. The badge means an audit exists and is accessible, not that the Store has endorsed the application's security.

Long-term support. Developers can declare a Long Term Support commitment in the application manifest by setting a support end date. The Store displays this date publicly on the application page. It is a voluntary public commitment with no automated enforcement, but the public visibility creates accountability. LTS applications are filterable in Store search for users who prioritize stability.

Profile package distribution. Signed environment profile packages (.lenv files, described in Section 8) are distributed through the Store using the same infrastructure as applications. Profile packages are free to publish and carry no commission.

8 Platform, Profiles, and Security

8.1 Account System

The Account System is the single place where the user manages all online accounts. Set up once, available everywhere, with full visibility into which application accesses what. The design principle is straightforward: no application ever directly handles OAuth flows, stores tokens, or manages credentials. All account logic lives in a dedicated *Account Daemon*.

Architecture. Applications declare what they need; the daemon handles the rest. When an application needs access to a service, it sends a request to the Account Daemon, which checks the application's permission profile, retrieves or derives the appropriate token, logs the access in the Audit Log, and returns what the application

needs. The application then communicates with the provider's API directly using that token. The daemon is the sole keeper of all credentials; tokens never live in application memory longer than necessary.

Service-level permissions. Applications request access at the service level, not the OAuth scope level. The user sees human-readable service names and never raw OAuth scope strings. When an application requests account access for the first time, the user sees exactly which service is being requested, grants or denies it, and the decision is stored in the application's unified permission profile as described in Section 7.3. Account permissions are part of the same system as filesystem, network, and graph permissions.

```
Google Account (personal@example.com)
  Mail -> allowed for: Thunderbird
  Calendar -> allowed for: Calendar App, AI Layer (read-only)
  Drive -> allowed for: File Manager
  Contacts -> not granted to anyone
```

Listing 8 Service-level permission view (Settings)

Multiple accounts and resolution order. When multiple accounts of the same type exist, the system resolves which to use via a three-tier model. The system default applies globally; an Isolated Desktop context can declare its own defaults, so that switching to the Work desktop automatically switches the active account context; and individual applications can be manually pointed at a specific account regardless of the desktop context. Resolution proceeds from most specific to least specific: per-app override, then desktop default, then system default.

```
# resolved in order: per-app override -> desktop default -> system default
[defaults]
google = "personal@example.com"

[desktop.work.defaults]
google = "work@example.com"

[app."org.mozilla.thunderbird"]
# always uses personal account regardless of active desktop
google = "personal@example.com"
```

Listing 9 Account resolution configuration

Token security. Credentials never leave the Account Daemon as-is. Two paths are used depending on provider support. For providers that support OAuth scope restriction, the daemon issues a short-lived derived token scoped only to what the application declared, with a default lifetime of five minutes. The full OAuth token and refresh token never leave the daemon at all. For providers that do not support downscoping, the full token is handed to the application but with two compensating controls: every token handoff is logged in the Audit Log, and tokens are actively revoked when the application process exits. All credentials at rest are stored in the Kernel Keyring;

the daemon loads them into memory only when actively needed and discards them immediately after.

Offline behavior. Two failure modes are handled distinctly. When the provider rejects the token because the user revoked access or the account password changed, the daemon emits a single system notification directing the user to Settings and does not repeat it; applications receive a clear `AccountAuthRequired` error code and surface a gentle in-app hint. When the network itself is unavailable, the daemon serves cached data from the last successful sync, clearly marked with the timestamp of the last update. A calendar application shows last-known events labeled with their age rather than an empty state. When connectivity is restored, the daemon refreshes silently in the background and notifies applications of updated data via the Event Bus.

GNOME Online Accounts bridge. Applications built for GNOME that use GNOME Online Accounts expect a D-Bus interface at `org.gnome.OnlineAccounts`. The system provides a GOA-compatible bridge that implements this interface exactly but sources all data from the Account Daemon internally. Evolution, GNOME Calendar, and other GOA-dependent applications work without modification. Internally every request goes through the Account Daemon, through the permission system, and through the Audit Log. The bridge translates; it does not add logic.

8.2 Profile System

A profile is a complete, isolated user environment backed by a separate Linux system user. Each profile has its own home directory, its own Knowledge Graph, its own account configuration, and its own set of running system daemons. Isolation is enforced at the kernel level, not by convention.

The profile system replaces the concept of isolated desktops within a session. Rather than trying to isolate individual desktops within a single session, the entire session is the isolation boundary. Switching profiles is a session switch, not a window toggle.

Personal profiles. The primary profile, created at system setup, cannot be deleted. It has no restrictions: full access to all installed applications, all accounts, and all system features. This is the default context the user logs into.

Work and custom profiles. A user-created or organization-deployed profile for a specific context such as work, a project, or a secondary identity. Backed by a separate Linux user with its own encrypted home image via `systemd-homed`. Virtual desktops, accounts, Knowledge Graph, and application configurations are all independent from the Personal profile. Multiple custom profiles can coexist. Each is a full Linux user; switching to one is a standard session switch with saved and restored window state.

Locked profiles. A profile deployed by an institution for controlled sessions: exams, regulated work environments. Backed by a temporary Linux user created at session start and deleted when the session ends. The user cannot switch to another profile

while a Locked session is active; this is enforced at the compositor level, not by UI alone. After the session ends, the Linux user and all associated data are removed. No data from the Locked session survives into the Personal profile.

Guest mode. A lightweight temporary context for short-term use by another person. Unlike other profile types, Guest does not create a second Linux user or a second compositor session. It runs as a sandboxed environment on top of the current session: a separate namespace, an empty home directory, no access to the host user's data. The host session is suspended in the background and resumed instantly when Guest ends. All Guest data is discarded on exit.

Technical foundation. Each non-Guest profile is a Linux system user. Profile creation is `useradd` plus `systemd-homed` home image setup; profile switching is a standard Linux user session switch; profile deletion is `userdel` plus home image removal. Isolation is kernel-enforced. `systemd-homed` provides per-profile LUKS-encrypted home images. When a profile is not active, its home image is unmounted and its encryption key is discarded from memory. Each profile runs its own instance of all system daemons: Graph Daemon, Account Daemon, Event Bus. There is no shared daemon state between profiles.

Data transfer between profiles. By default profiles are fully sealed from each other. Controlled data transfer is configured per profile pair in Settings, with independent directional control over clipboard, drag-and-drop, and a shared folder:

```
[profile.work.transfer]
clipboard = "in-only" # "off" | "in-only" | "out-only" | "both"
drag_drop = "in-only"
shared_folder = "~/Transfer/Work"
shared_folder_permission = "read-write" # or "read-only" | "off"
```

Listing 10 Cross-profile transfer configuration

A dedicated *Transfer Daemon* is the only process with simultaneous access to both profile namespaces. All cross-profile data flow goes through it; nothing bypasses it. Whenever data flows out of a profile, a non-dismissable toast appears for three seconds identifying the source profile, destination, and file name, with an Undo option. The transfer proceeds but can be cancelled within the window. All transfers are recorded in the Audit Log with source, destination, transfer type, and data description.

Locked profiles have transfer hardcoded to off in all directions. This cannot be overridden by the user or by an organization's configuration package; it is a system-level invariant with no configuration surface.

Signed profile packages. Profiles can be deployed via a *Signed Profile Package*, a `.lenv` file created and signed by an organization. The user installs it themselves and retains full control over their device. The organization gets a tamper-evident, versioned configuration. This is the BYOD model: the organization controls their context, the user controls their device.

```
[environment]
id = "com.acme.work-profile"
name = "Acme␣Work"
version = "2.1.0"
publisher = "Acme␣GmbH"
type = "persistent" # or "locked" for exam/controlled sessions

[transfer]
clipboard = "in-only"
drag_drop = "in-only"
shared_folder = "off"

[network]
require_vpn = "acme-vpn"
allow_outbound = ["*.acme.com", "*.office365.com"]
block_outbound = ["*"]

[apps]
required = ["com.acme.timetracker", "org.mozilla.firefox"]
blocked = ["com.valve.steam"]

[graph]
ai_access = "off"
export = "off"
```

Listing 11 Signed profile package format

The package is signed by the organization's private key. Key distribution uses TOFU by default: on first install the user sees the publisher name and key fingerprint and confirms trust. For managed onboarding, IT can send an enrollment link so that the organization's key is imported before any package arrives, eliminating the TOFU prompt entirely.

Before installation, a full summary of every policy and restriction is shown. Nothing is hidden. There are certain things an organization cannot do through a signed package, enforced technically rather than by policy: the organization cannot read the user's Personal profile or its Knowledge Graph, cannot access any data outside the profile's Linux user boundary, cannot monitor activity in other profiles, cannot remotely wipe or modify the device, and cannot prevent the user from uninstalling the package. If notify_on_uninstall was declared and accepted at install time, the organization receives a notification when the package is removed, but the removal proceeds regardless.

8.3 Gaming and Windows Compatibility

Gaming and Windows application compatibility are first-class features, not afterthoughts. The required components are pre-installed and pre-configured; no manual setup is required.

Wine and Proton. *Wine* provides compatibility for Windows applications on Linux. *Proton* is Valve's fork of Wine, optimized for gaming and maintained with extensive

patches from Valve's Linux gaming team; for gaming specifically it is substantially better than vanilla Wine. Both are pre-installed alongside *DXVK*, which translates Direct3D 9, 10, and 11 calls to Vulkan, and *vkd3d-proton*, Valve's fork of vkd3d that translates Direct3D 12 calls to Vulkan. Winetricks is included for installing Windows runtime components into Wine prefixes.

Steam is available in the Store and works out of the box with Proton enabled. Steam manages its own Proton version per game; the system does not interfere with Steam's internal Proton management.

Non-Steam Windows games and applications use Wine directly, managed through the first-party *Wine Manager* application. The underlying components are shared with Steam's installation rather than duplicated. Wine Manager is a thin UI over the existing tooling providing prefix creation and deletion, per-prefix Winetricks access, launch shortcuts, and per-prefix Proton and Wine version selection.

DXVK and vkd3d-proton require Vulkan. Mesa provides Vulkan support for AMD and Intel GPUs by default. Nvidia requires the proprietary driver for Vulkan support; the Store provides explicit guidance for Nvidia users during setup.

Wine theming. By default Wine applications render with a Windows Classic or Windows 7-era appearance that is visually inconsistent with a modern desktop. A first-party Wine theme is maintained as part of this project, generated from the same design token system used for GTK and Qt theming. It updates automatically when the system theme changes. Wine applications do not look native, but they look consistent with the rest of the system. The Wine theme is applied automatically to all Wine prefixes managed by the system; users who prefer the default Windows appearance can opt out per prefix.

Performance tooling. *Gamemode* by Feral Interactive is pre-installed and enabled by default. It applies system-level performance optimizations when a game is running: the CPU governor switches to performance mode, the I/O scheduler is tuned, and kernel scheduler hints are applied. Proton handles the Gamemode activation call automatically; no manual configuration is required.

MangoHud is pre-installed as an in-game overlay showing frames per second, GPU and CPU usage, temperatures, and frame timing. It is disabled by default and toggled via a keyboard shortcut.

Knowledge Graph integration. Gaming activity is tracked in the Knowledge Graph like any other activity: which games were played, when, and for how long; which Wine and Proton versions were used per game; and performance data from MangoHud when enabled, linked to sessions. This makes queries such as "how many hours did I play last week" or "which Proton version worked best for this game" available locally without relying on any external service.

8.4 Security and Privacy

Security is not a feature added on top of this system. It is a foundational design constraint that shapes every architectural decision.

Core principles. Five non-negotiables drive everything in this section. First, the Knowledge Graph, Audit Log, and AI context are local by default and never leave the machine without explicit user action; there is no opt-out dark pattern, the default is that nothing goes anywhere. Second, the system collects no usage data, crash reports, or any other information without explicit user consent. Third, the system works fully offline and without registration; cloud features are opt-in. Fourth, every component gets exactly the permissions it needs and nothing more, applying to filesystem access, network access, graph access, and syscalls alike. Fifth, security must not destroy usability: defaults are chosen to protect without requiring constant interaction, because a system so locked down that users disable its protections is less secure than a well-calibrated one.

A sixth principle follows from the others: data minimization is itself a security property. Data that does not exist cannot be stolen. The retention policy defined in Section 4.5 is not only a privacy decision but an active security measure. Every decision to extend retention windows is a deliberate expansion of attack surface and must be understood as such.

8.4.1 Full Disk Encryption

Full Disk Encryption means that everything stored on disk is encrypted at rest. On Linux this is implemented via *LUKS* on top of the *dm-crypt* kernel module. The entire partition is encrypted and the OS decrypts it transparently at boot after credentials are provided.

FDE is enabled by default. A stolen or lost device is a realistic threat for any user; without FDE, physical access means full access to all data. The performance concern that historically argued against FDE no longer applies: all modern x86 and ARM processors have hardware AES acceleration that makes encryption effectively free at current workloads.

The primary unlock mechanism is a passphrase entered at boot. On systems with a *TPM*, the FDE key can be sealed to the TPM and released automatically if the system boots in a known-good state. This allows passwordless boot on trusted hardware while still protecting the drive if it is physically removed. If the boot configuration changes due to tampering, the TPM refuses to release the key and falls back to the passphrase. TPM-based unlock is opt-in; passphrase is always the fallback.

Hibernate requires additional care because the entire RAM contents are written to the swap partition. That swap partition must also be LUKS-encrypted, and key management must allow the system to resume without manual intervention. This is a known-solved problem on Linux but requires deliberate configuration during the installation process.

8.4.2 Secure Boot, Unified Kernel Images, and Measured Boot

Secure Boot is a UEFI firmware feature that verifies the cryptographic signature of every piece of code executed during the boot process. If any component is not signed by a trusted key, the firmware refuses to execute it. The practical effect is that an attacker who modifies the bootloader or kernel cannot boot the modified system without also possessing the signing key.

Both candidate base distributions support Secure Boot via a Microsoft-signed shim bootloader. Custom kernel modules built for this project must be signed with a Machine Owner Key enrolled in the firmware.

A traditional Linux boot chain consists of multiple separate files: the bootloader, the kernel image, and the initramfs. A *Unified Kernel Image* packages all three into a single signed EFI binary, reducing the attack surface to a single component requiring verification. `systemd-boot` supports UKI natively and the broader Linux ecosystem is moving in this direction. This project targets UKI as the boot format.

Measured Boot goes further than Secure Boot: every boot component is cryptographically hashed and the hash is stored in the TPM's Platform Configuration Registers. This creates a tamper-evident record of exactly what ran during boot. If any component changes, even legitimately due to an update, the PCR values change. Locally, the TPM can be configured to release the FDE key only if the PCR values match an expected set, so that a modified bootloader causes the TPM to refuse key release. In managed environments, a remote server can request a signed attestation report from the TPM, and a device that fails attestation can be denied network access.

Secure Boot combined with *Measured Boot* makes the Evil Maid Attack significantly harder. An Evil Maid Attack is the scenario where an attacker has brief physical access to a powered-off device and modifies the bootloader to capture the user's FDE passphrase on the next boot. Without the signing key, Secure Boot blocks the modified bootloader from executing entirely.

8.4.3 App Sandboxing

Every application runs in a restricted environment that limits what it can do even if fully compromised. Three mechanisms operate simultaneously and independently; bypassing one does not bypass the others.

Table 17 Sandboxing mechanism layers

Mechanism	Function
Linux Namespaces	Changes what the application can see
AppArmor	Blocks actions on resources the application can see
seccomp	Blocks specific syscalls regardless of resource visibility

Linux Namespaces give each application its own restricted view of the system before any permission check occurs. A Mount Namespace means the application sees only its own data directory, explicitly granted paths, and a minimal set of system paths; other applications' data directories do not exist from its perspective. A Network Namespace means applications without declared network access see no network interface at all;

applications with declared network access see only a filtered virtual interface. A PID Namespace means the application can only see its own process tree.

AppArmor profiles are defined externally in the package and reviewed during Store submission; an application cannot define or modify its own profile. A text editor does not need ptrace, mount, or kexec; a seccomp profile that excludes these syscalls causes the kernel to kill the process immediately if a compromised application attempts them.

IPC is restricted to the Event Bus, the Graph Daemon, and explicitly declared inter-application communication channels. An application cannot open a socket to another application without that channel being declared and approved.

8.4.4 Knowledge Graph Security

The Knowledge Graph is the most sensitive component in the system. It accumulates a detailed record of everything the user does and its access control must be correspondingly rigorous.

No application has direct access to the Kuzu database files. All graph access goes through the *Graph Daemon*, a privileged system process that is the sole reader and writer of the graph. Every other component communicates only with the daemon over a defined IPC interface, and the daemon checks permissions before executing any query.

Three axes of permission. Graph permissions are defined along three independent axes. The first axis is node types: which categories of data the caller can read, such as File, App, NetworkConnection, or Session. A calendar application may read Session nodes but not NetworkConnection nodes. The second axis is relations: which connections between nodes the caller can traverse. An application permitted to read File nodes may still be blocked from traversing the OPENED_BY relation that connects files to the applications that opened them. The third axis is instances: whether the caller can see only the nodes it generated itself or nodes generated by other applications as well.

```
[app."com.example.notes"]
allow_read = ["File.path", "File.last_modified", "Session.started_at"]
deny_read = ["File.content_snippet", "NetworkConnection.*"]
allow_relations = ["PART_OF_SESSION"]
instance_scope = "self" # only nodes this app generated
```

Listing 12 Graph permission entry

Two permission layers. Permissions are assigned in two layers. At install time, the package declares the maximum graph permissions it can ever receive; the user reviews this declaration as part of installation. At runtime, the application can request permissions within its declared maximum, with a user prompt explaining what is requested and why. A runtime request that exceeds the install-time declaration is rejected outright. Some fields within permitted node types are additionally restricted regardless of node-level permissions: file content snippets, for example, require an explicit extra permission separate from general File node access.

Kuzu file protection. The Kuzu database files are protected by three independent layers. At the filesystem layer, the Graph Daemon runs as a dedicated system user graph-daemon and the database files are owned exclusively by this user with 600 permissions. A successful sandbox escape by an application process running as the logged-in user does not grant access to these files. At the AppArmor layer, a separate deny rule in every application's AppArmor profile prohibits access to the graph directory even if filesystem permissions were bypassed. At the encryption layer, the graph directory lives inside a LUKS-encrypted loopback image that the daemon mounts at startup using a key retrieved from the Kernel Keyring. When the daemon stops, cleanly or via crash, the image is unmounted and the key is discarded from memory. During any period when the Graph Daemon is not running, the Kuzu files are LUKS-encrypted ciphertext unreadable even to root.

Cypher injection prevention. The Graph Daemon never accepts query strings with embedded values. All queries must use named parameters, and the daemon runs a validator on every incoming query that detects embedded values before execution.

```
// Rejected - string interpolation
let query = format!("MATCH (f:File {{name: '{}'}}) RETURN f", user_input);

// Required - parameterized
daemon.query(
    "MATCH (f:File {name: $name}) RETURN f",
    params!{ "name" => user_input }
)
```

Listing 13 Parameterized graph queries

Applications without arbitrary_query permission in their manifest cannot send free-form query strings at all; they select from pre-approved query templates and supply only the parameters. Only explicitly trusted components such as the AI layer and system daemons receive arbitrary_query permission, and even they go through the parameterization validator.

Capability tokens. Instead of looking up permissions in an ACL table on every query, applications receive a cryptographically signed capability token at startup. The token encodes the application's allowed scopes directly, and the daemon verifies the HMAC signature on each query without a database lookup. This is faster at high query rates, more robust against a corrupted ACL table, and allows the AI layer to derive restricted sub-tokens for specific query scopes without involving the daemon. Token revocation when permissions change mid-session is handled via a small in-memory revocation list that is cleared on daemon restart.

Graph Daemon hardening. The daemon runs with the minimum privileges needed for its function, enforced by a systemd service unit with PrivateNetwork=true, MemoryDenyWriteExecute, PrivateTmp, and a tightly scoped SystemCallFilter. PrivateNetwork=true means a compromised daemon cannot make outbound connections to exfiltrate data. If the daemon is compromised despite these controls, the blast radius is bounded to

Kuzu data only, and a systemd watchdog restart closes the LUKS image cleanly and starts fresh.

Clients verify that they are speaking to the legitimate Graph Daemon before sending queries. At startup, the daemon signs a challenge token with a private key that only it holds; clients verify the signature against the public key distributed with the system package. A connection that fails verification is dropped, preventing a trojanized replacement daemon from intercepting graph traffic.

Rate limiting and query complexity. Two attack vectors are addressed: denial of service via query flooding, and timing side-channels. Rate limiting is enforced per client identity using a token bucket: normal applications are limited to 100 queries per second with a burst limit of 200; the AI layer has a higher sustained limit appropriate to its workload; system daemons are unlimited. Exceeding the rate limit generates an event for the Anomaly Detector, and repeated violations trigger an alert.

Query complexity limits prevent expensive traversals from being used as denial of service. Traversal depth is capped at 5 hops by default and never more than 10. Every query must include a `LIMIT` clause; queries without one are rejected before execution. Queries are aborted after 500 milliseconds; the client receives `QueryTimeout`. Response times have a small random noise value added to make statistical timing attacks on the graph impractical.

8.4.5 AI Security

The AI layer has broader graph access than most applications by necessity, making it a higher-risk component with its own security model.

AI access levels. The user configures a global AI access level as the primary control over how much the AI can see. Table 18 lists the five levels. Action permissions, meaning what the AI can do rather than what it can read, are configured separately. Reading data and taking actions are fundamentally different risk categories and are therefore independently controlled. Suggest only mode means the AI proposes actions and the user manually executes them. Supervised mode means the AI executes actions but shows a preview with a cancellation window before proceeding. Autonomous mode means the AI can act within specific applications without per-action confirmation, and each application must be individually enabled for this; autonomous mode is never a global setting.

Prompt injection. Prompt injection is an attack where malicious instructions are embedded in content the AI processes with the intent of manipulating its behavior: a PDF might contain hidden text instructing the model to send files to an external server. Current language models cannot reliably distinguish between content they should process and instructions they should follow. This is a structural limitation, not a fixable bug. The defense is therefore architectural.

Every piece of content the AI receives is tagged with its origin before entering the prompt, so the model sees explicit markers distinguishing user input, external content such as PDFs, and graph data. Before external content reaches the main model,

Table 18 AI read access levels

Level	Name	What the AI can access
0	Minimal	Nothing from the graph; only what the user explicitly pastes
1	Session-scoped	Only data from the current active session
2	Project-scoped	All data associated with a user-selected project
3	Time-scoped	Historical data up to a configurable lookback window
4	Full read	Everything the user has read access to

it passes through a small local model whose sole function is to detect AI-directed instructions; this is a probabilistic first pass that catches obvious injection attempts cheaply and locally.

As a hardcoded rule with no configuration surface, any action the AI wants to take that was triggered by external content requires explicit user confirmation regardless of the configured action permission level. An AI that read a malicious PDF cannot use that as a basis to execute commands without the user seeing exactly what it wants to do and why.

Documents are parsed in an isolated subprocess before their text content is passed to the AI. The subprocess has no network access and no access to the graph; only the extracted text leaves the sandbox, with no embedded scripts, no active content, and no metadata that could carry instructions.

AI network proxy. The AI layer has no direct network access. All cloud API calls go through a dedicated system proxy daemon that is the only component allowed to make outbound connections to AI provider endpoints. The AI constructs a request, hands it to the proxy, and receives only the response. This prevents a compromised AI layer from independently exfiltrating data over arbitrary network connections, ensures all AI-related network traffic is logged and attributable, and allows the proxy to enforce that only approved provider endpoints are contacted.

8.4.6 Audit Log

Every access to the Knowledge Graph, every AI action, and every permission grant or denial is recorded in the Audit Log. This serves two purposes: giving the user visibility into what is happening on their system, and providing the data needed for behavioral anomaly detection.

The Audit Log is an append-only ledger. Entries cannot be modified or deleted; only new entries can be added. Each entry is cryptographically linked to the previous one via a hash chain, so tampering with any historical entry is detectable by any reader that verifies the chain.

Two log tiers coexist. The Structural Log is always active and records interaction metadata without content: timestamp, application identity, which node types were queried, which relations traversed, result count, duration, and status. The query string itself, result contents, and concrete node IDs are never recorded in this tier. This is

sufficient to detect suspicious behavior without being sufficient to reconstruct what data was accessed.

The Forensic Log is opt-in and time-limited. When activated in Settings, it additionally records the full query string, query parameters (not result contents), and the calling process stack trace. Forensic Mode is always limited to a maximum of 24 hours and then automatically deactivates. It is shown prominently as active in Settings while enabled and cannot be activated remotely by any application or administrator; only the user, locally.

A dedicated Audit Daemon is the sole writer to the log files. Other components send audit events over a restricted IPC channel and cannot write directly to the log. The Forensic Log is accessible exclusively to the user; no administrator, no daemon, and no application can read it. The Structural Log is readable by the user and by the Anomaly Detector for pattern analysis; in managed environments, administrators receive aggregated anomaly alerts only, never raw log contents.

8.4.7 Anomaly Detection

The Anomaly Detector is a system daemon that reads the Audit Log and identifies unusual patterns. It is not a traditional antivirus and does not match file signatures or scan for known malware. It is a behavioral analysis tool that establishes a baseline of normal behavior on this specific machine and flags deviations.

Signature-based detection requires a continuously updated database of known threats and cannot detect novel attacks. Because this system already maintains a rich picture of normal behavior via the Knowledge Graph and Audit Log, behavioral detection is the natural fit: the data needed to define normal behavior already exists.

The Anomaly Detector watches for patterns including an application suddenly querying graph node types it has never queried before; query volume from an application spiking above its historical baseline; an AI action executed without a preceding user interaction in the same session; a network connection from a process that has never previously made network connections; a new eBPF program being loaded outside system update windows; and permission escalation requests at unusual times.

Alerts are surfaced via the notification system with enough context for the user to make a decision: which application, what it did, why it is unusual, and options to investigate or dismiss. The detector does not automatically block anything; it informs. Automatic blocking based on anomaly scores is an opt-in feature for managed environments where an administrator configures the policy. The Anomaly Detector runs locally and alert data does not leave the machine unless the user or administrator explicitly configures remote alert forwarding.

8.4.8 Network Security

On a conventional Linux system, any process can open a network connection to anywhere by default. This is the wrong default. On this system, outbound network access is denied unless explicitly permitted. Every application that needs network access declares it in its package manifest by protocol, destination, and direction; connection attempts outside the declaration are blocked at the kernel level via eBPF-based network filtering and logged as security events.

VPN enforcement can be configured in managed environments such that certain network destinations are reachable only when a VPN is active. WireGuard is the preferred VPN protocol given its in-kernel availability since Linux 5.6, clean key management, and performance characteristics. OpenVPN support is provided for compatibility with existing enterprise infrastructure.

8.4.9 Physical Security

Full Disk Encryption handles the most common physical threat, but physical access threats go further.

A Cold Boot Attack exploits the fact that DRAM does not lose its contents instantly when power is removed. At low temperatures, an attacker who obtains a running or recently suspended device could potentially read the FDE key from memory. Two mitigations apply: the kernel is configured to overwrite RAM contents before suspend-to-disk, and AMD SME and Intel TME are hardware features available on modern CPUs that encrypt the entire RAM contents with a key that lives inside the CPU die and never leaves it. Even if an attacker physically removes the RAM modules, they read only ciphertext. Both are enabled by default where hardware supports them.

USBGuard enforces a policy on USB device connections. When the screen is locked, no new USB devices are permitted to become active; only devices connected before the screen locked remain accessible. This blocks BadUSB attacks, where a specially crafted USB device presents itself as a keyboard and injects keystrokes. Enabled by default; specific devices such as hardware security keys can be whitelisted in Settings.

IOMMU restricts which memory regions each hardware device can access via Direct Memory Access. With IOMMU enabled, a malicious Thunderbolt or PCIe device can only access the memory regions the OS has explicitly mapped for it, not arbitrary system RAM. Enabled by default via kernel parameters. For Thunderbolt specifically, the default security level requires newly connected devices to be explicitly authorized by the user before they are activated; previously authorized devices are remembered.

TPM Remote Attestation allows a device to cryptographically prove to a remote server that it booted in a known-good state by sending a signed report of its measured boot PCR values. In a managed deployment this can gate network access: a device that fails attestation is denied VPN access until an administrator investigates. This is inspired by the BeyondCorp model and represents a meaningful differentiator for enterprise deployments.

8.4.10 Memory Safety

Memory safety bugs are the most common source of exploitable vulnerabilities in system software and are structural problems with C and C++ that persist even with experienced developers. All system components developed for this project are written in Rust. The Rust compiler enforces memory safety at compile time through its ownership and borrow checker; an entire class of bugs that would be possible in C simply cannot compile in Rust. This is a compile-time guarantee, not a runtime check with overhead.

Third-party applications cannot be rewritten. Sandboxing limits what a compromised application can do; hardware-level mitigations provide a further layer. Intel

CET adds a hardware-enforced shadow stack, a separate protected copy of return addresses that the CPU verifies on every function return. Control-flow hijacking attacks that overwrite return addresses produce a mismatch that causes an immediate fault. This is supported by the Linux kernel since version 6.6 and is enabled by default for new processes on supported hardware with no application changes required. For any C or C++ components compiled as part of this project, Clang's CFI instrumentation is applied, validating at runtime that indirect calls and virtual dispatch target only legitimate destinations.

8.4.11 Multi-User Isolation

When multiple users share a system, their isolation goes deeper than traditional Unix file permissions because the Knowledge Graph, AI configuration, and Audit Log introduce sensitive state beyond the home directory. Each user has a separate Kuzu instance, a separate Graph Daemon process running under their own system user, and a separate Audit Log. These are not shared resources with access control layered on top; they are separate instances that other users' processes cannot reach.

Home directories are managed via `systemd-homed`: each user's home directory is stored as a LUKS-encrypted portable image. When a user logs out, their home image is unmounted and the encryption key is discarded from memory. No other user, including root, can access the home image without the user's credentials. The Knowledge Graph lives inside the home directory and is therefore protected by this encryption as a structural consequence rather than an additional policy.

In a multi-user or managed environment, an administrator can install and remove system-wide software, configure network policies, set password and screen lock policies, view aggregated anomaly alerts, manage USBGuard whitelists, and manage Secure Boot keys. The administrator explicitly cannot read any user's Knowledge Graph, access any user's home directory contents, view any user's Audit Log, or intercept any user's AI sessions. These limitations are enforced by `systemd-homed` and the per-user Graph Daemon architecture, not by policy alone. The administrator sees that something unusual happened via anomaly alerts; they do not see what.

8.5 Managed Environments

For enterprise deployments where this system runs on many machines administered centrally, additional infrastructure is required beyond what individual Signed Profile Packages provide. This section describes the managed environment architecture for organizations that need fleet-level control.

Policy Engine. A system daemon receives signed policy packages from a central management server and applies them locally. The daemon verifies the cryptographic signature of every package before applying any policy; unsigned or incorrectly signed packages are rejected outright. Policies can control permitted and blocked applications, network access rules beyond per-app declarations, AI provider restrictions including complete disabling of cloud AI, the USBGuard device whitelist, screen lock timeout and password complexity requirements, update scheduling so that updates are applied

only during configured maintenance windows, and a minimum audit log retention period for compliance purposes.

LDAP and Active Directory. Most enterprises already have an identity management system, typically Microsoft Active Directory or an LDAP-compatible directory. Integration with these systems is a baseline requirement for enterprise adoption. User authentication, group membership, and basic policy delivery are handled via standard LDAP and Kerberos protocols that existing enterprise infrastructure already supports, requiring no proprietary agents on the management server side.

MDM-style management. Beyond LDAP, a modern MDM-style management layer provides richer capabilities: device enrollment, remote policy push, health status reporting, and remote wipe. This is a longer-term goal; LDAP compatibility is the foundation and MDM-style management is built on top of it once the base is stable.

Remote attestation as network gate. TPM Remote Attestation, described in Section 8.4.9, can be used as a network access gate in managed deployments. The workflow is:

1. The device boots and generates a TPM attestation report containing signed PCR values.
2. The device connects to the management server's attestation endpoint.
3. The server verifies the report against the expected baseline for that device's configuration.
4. If verification passes, the device receives its VPN credentials and network access.
5. If verification fails, the device is quarantined pending administrator review.

This ensures that a device with a modified bootloader, disabled Secure Boot, or an outdated security policy cannot silently access company resources. It is inspired by the BeyondCorp model and is a meaningful differentiator for enterprise deployments.

GDPR and compliance tension. In managed environments there is an inherent conflict between user privacy rights and organizational compliance requirements. GDPR grants users rights over their personal data; many compliance frameworks in financial services, healthcare, and legal contexts require organizations to retain logs and audit trails. This system does not resolve the conflict, because it is a legal and organizational question rather than a technical one.

What the system does is make both sides technically possible. Users retain full visibility into and control over their own Knowledge Graph and personal Audit Log. Organizations can configure a minimum retention period for compliance-relevant audit events, which do not include Knowledge Graph contents. The separation between personal graph data, which is private to the user, and system audit events, which may be subject to organizational retention requirements, is maintained architecturally rather than by policy. Organizations deploying this system should seek legal counsel to define appropriate policies. The system provides the technical primitives; the legal framework is out of scope.

9 Companion App and Health

9.1 Companion App

The system ships with a first-party companion app for Android and iOS that bridges phone and desktop into a single coherent experience. It is built with Tauri 2.0 using the same Rust backend, the same design token system, and the same UI component library as the desktop. The companion app looks and feels like the OS because it is built from the same parts.

The protocol is custom. KDE Connect was evaluated and rejected: its protocol design would obstruct the deeper integration features planned for later tiers, and building on top of it would mean working around its assumptions rather than designing for the actual requirements.

Feature tiers. Features are organized into three trust tiers established at pairing time. Table 19 shows the breakdown. The distinction between paired and trusted exists because Tier 2 features involve access to graph context and health data, which require a higher trust level than file transfer and notification mirroring. Tier 2 is restricted to the official app; a third-party client that implements the pairing protocol only gets Tier 1.

Table 19 Companion app feature tiers

Tier	Trust level	Features
1	paired	File Drop; clipboard sync (manual trigger only, not automatic); notification mirroring; media playback control
2	trusted	Handoff (continue reading, browsing, or editing across devices); Unified Timeline (cross-device activity, local only); graph presence (phone sends active app and content context); health data forwarding
3	Planned	Wearable data pipeline; extended context sharing

Protocol and transport. Discovery uses mDNS service advertisement on a shared WLAN. Without a shared network, the desktop and phone locate each other via BLE presence broadcast carrying the device ID. Transport is selected by availability at connection time, shown in Table 20.

Table 20 Companion app transport selection

Transport	Use case
HTTPS over LAN	Primary transport; full feature set
WiFi Direct	Large transfers without a shared router
Bluetooth	Small data only: clipboard, notifications, control commands

Pairing uses a Trust on First Use model. Both devices generate an Ed25519 keypair on first launch. The first connection uses TLS with self-signed certificates plus a six-digit confirmation code displayed on the desktop and confirmed on the phone. After pairing, reconnection is automatic with no further confirmation. Unpair deletes the key on both sides with no trace.

Auto-connect. Paired devices broadcast BLE presence in the background. When a known device is detected, the system connects automatically using the best available transport. No user action is required after initial pairing. Android supports full background BLE advertising reliably. iOS imposes CoreBluetooth Background Mode restrictions: auto-connect works but is less reliable and carries a marginally higher battery cost on the iPhone side.

Theme synchronization. Because the companion app uses the same design token system as the desktop, the active theme propagates to the phone automatically: accent color, dark and light mode, typography. This is a structural side effect of the shared token infrastructure, not a feature that was separately implemented.

iOS limitations. iOS imposes constraints that cannot be worked around. There is no persistent background BLE scanning, which makes auto-connect less reliable and requires more manual reconnects than on Android. WiFi Direct is not available to third-party iOS apps. USB-C is inaccessible to third-party apps without MFi certification. Health data forwarding via HealthKit is feasible with explicit user permission.

Tier 1 works well on iOS. Tier 2 is functional but degraded. iOS is explicitly a secondary platform and this limitation is communicated honestly in the app and in documentation rather than obscured.

9.2 Health and Wellbeing

The system includes a first-party health data component: a local daemon that aggregates, stores, and optionally analyzes health and wellbeing data from connected sources. This is not a fitness application. It is health data infrastructure: an open, local equivalent of Apple Health or Google Fit, built as a system component with the same privacy guarantees as the rest of the platform.

Data architecture. Health data lives in a dedicated local store managed by a *Health Daemon*, separate from the Graph Daemon. Data never leaves the device without an explicit user action. Clinical data uses FHIR R4 (HL7 Fast Healthcare Interoperability Resources) where applicable, enabling import from hospital and clinic systems, which are increasingly FHIR-capable across the EU, and export to FHIR-compatible tools without conversion. Non-clinical data such as steps, sleep stages, heart rate variability, and nutrition uses an open JSON schema with documented field definitions. There is no proprietary format; full export is always available.

Data sources. The initial release supports manual input and phone sensors forwarded via the Companion App: steps, GPS, and screen time. The plugin-based architecture is designed from the start to accommodate further sources without structural changes: wearables via the Companion App, hospital and clinic FHIR import, and third-party applications via a documented Health API gated by the standard permission system.

Wearable pipeline. Wearables connect to the system through the Companion App rather than directly, using the wearable manufacturer's existing phone SDK on the phone side. The data path is:

```
Wearable -> Companion App (phone, manufacturer SDK)
         -> OS Health Daemon (via Companion protocol)
```

Listing 14 Wearable data path

No direct wearable protocol implementation is required in the OS. The Health Daemon's plugin interface is designed to accept forwarded wearable data from the start, so that adding wearable sources later does not require structural changes. This is planned architecture, not initial release scope.

Graph integration. Health data integration with the Knowledge Graph is strictly opt-in and split into three explicit levels, all of which default to off. Table 21 lists the levels. Each level requires the previous one; the Settings UI enforces this and does not allow incompatible states. The AI layer's health-adjacent features are gated entirely on these toggles. With all three off, the health module is a pure local viewer and aggregator with no AI involvement whatsoever.

Table 21 Health graph integration levels

Level	What it enables
1 - Health Nodes	Health events appear as standalone nodes in the Knowledge Graph
2 - Correlations	Graph links health data to activity data by time window (e.g. sleep quality correlated with focus session length)
3 - Pattern Recognition	Full cross-domain analysis across health, productivity, and communication data

Settings and permissions. Health has its own dedicated section in Settings rather than being buried under general privacy settings. Controls include per-source enable and disable for each data source independently; per-data-type enable and disable so the user can, for example, share step data with an app but not sleep data; the graph integration toggles from Table 21 presented as a grouped sub-section with clear explanations of what each level enables; a configurable data retention period per data type; full FHIR and JSON export; and full deletion with a confirmation step.

Third-party applications request health data access per data type through the standard permission system, but health-specific confirmation dialogs are more explicit

about data sensitivity than generic permission prompts. A fitness app requesting sleep data gets a dialog that names the data type and explains clearly that this is health information, not a generic file access request.

10 Developer Experience, Sustainability, and Roadmap

10.1 Developer Experience and Infrastructure

The development infrastructure is designed around one constraint: a project of this scope spans many components with cross-cutting concerns, and the tooling must make that manageable without forcing artificial coupling between parts that should evolve independently.

Repository structure. The project uses a multi-repo layout under a single GitHub organization. Each component has its own repository, its own release cycle, and its own CI pipeline. Components can be updated independently as long as they do not introduce breaking changes to the interfaces they expose. A breaking interface change requires a coordinated update across all affected components, enforced by the versioning mechanism described below.

```
org/
  blueprint <- architecture documents and this document
  kernel-layer <- eBPF programs, Event Bus daemon
  knowledge <- Kuzu integration, graph schema, Graph Daemon
  ai-layer <- provider abstraction, MCP client
  compositor <- Smithay-based Wayland compositor (Rust)
  desktop-shell <- taskbar, launcher, shell (Tauri + TypeScript)
  ui-kit <- shared component library (TypeScript + Tailwind)
  os-sdk <- shared Rust crate for system integration
  app-files <- File Manager
  app-settings <- Settings
  app-terminal <- Terminal
  app-store <- Store
  module-sdk <- SDK for shell module development
  themes <- GTK4, Wine .msstyles, token source
  distro <- build system, ISO generation, meta-packages
```

Listing 15 Repository structure

Build system. *mkosi*, maintained by the systemd team, is the target tool for image generation. It builds bootable OS images from a declarative configuration, specifying which packages to include, which files to add, and how to configure the system. It produces UEFI-bootable images directly and supports multiple output formats: ISO, disk image, and container. It integrates cleanly with the Open Build Service. Confirming that mkosi handles the full build pipeline in practice is an open item pending actual use.

The *Open Build Service* handles packaging for all custom components. OBS builds RPM packages across architectures for both candidate base distributions, signs packages with the project GPG key, and hosts the package repository that end-user systems pull from. Both Fedora and OpenSUSE targets can be built from the same spec files, which is one practical reason both remain viable candidates.

Update system. Updates arrive from two independent streams managed separately. The base system, meaning the kernel and base distribution packages, is handled by the native package manager (dnf or zypper depending on the chosen base), with Snapper automatically taking a Btrfs snapshot before every update. Project components, meaning the compositor, shell, event bus, graph daemon, AI layer, and first-party applications, are distributed via the project's own OBS repository as RPM packages managed as a coordinated meta-package.

The update behavior is designed to be non-intrusive. Security updates for both streams are downloaded automatically in the background and applied at the next shutdown or reboot, never during an active session. No forced reboots, no countdown timers. All other updates are downloaded silently in the background; a single calm notification appears when they are ready. The user applies them when convenient. In managed environments, administrators can configure mandatory update windows; even then, no mid-session interruption occurs.

Rollback is available for every update: Btrfs and Snapper record a snapshot before each change, and the user can boot into a previous snapshot from the boot menu and roll back the entire system without any additional tooling.

Project components communicate over defined IPC interfaces. A breaking interface change increments the major version number and must be shipped as a coordinated meta-package update, so that incompatible component versions cannot coexist. Components verify interface compatibility at startup:

```rust
fn check_compatibility() -> Result<()> {
    let daemon_version = get_graph_daemon_version()?;
    if daemon_version.major != EXPECTED_MAJOR {
        bail!("Graph Daemon interface version mismatch. Run system update.");
    }
    Ok(())
}
```

Listing 16 Interface compatibility check at startup

A component that detects a version mismatch refuses to start and surfaces a clear message. The only resolution is applying the pending system update.

CI/CD pipeline. Tests run at three levels of integration. Table 22 shows when each level runs.

Unit tests run per repository against mocks from `os-sdk`. Rust components use `cargo test`; TypeScript components use Vitest. No PR merges without green unit tests. The `kernel-layer` is a special case: eBPF logic that can be tested without a real kernel runs on the host as normal Rust unit tests, while eBPF programs themselves run in a minimal QEMU instance that CI spins up automatically.

Table 22 CI trigger summary

Trigger	What runs
Every push or pull request	Unit tests for the affected repository
Pull request merge to main	Unit tests, lint, and build check
Daily at 02:00	Cross-component integration tests against all main branches
Release candidate	Full VM tests and smoke tests
kernel-layer push	Unit tests plus VM-based eBPF tests

Cross-component integration tests verify that two or more components communicate correctly over their real IPC interfaces. They run on the host without a full system boot, starting only the directly involved daemons as real processes against real IPC sockets. A failing integration test with all unit tests passing means interface drift between repositories, which is the primary failure mode this level exists to catch. Examples: the Event Bus emits a `FileAccessEvent` and the Graph Daemon writes correctly to Kuzu; a theme token change in Settings propagates to all running Tauri processes; the shell sends a keybinding action and the Input Manager calls the correct handler.

Full VM tests boot a complete system image in QEMU and run smoke tests: system boots without crash; Spotlight opens, finds an application, and launches it; eBPF events arrive and land in the Knowledge Graph; theme switch propagates correctly; module install and uninstall behave as expected. These tests are slow and run on release candidates only. The VM image is built by the same script used for real releases; no separate test image exists.

Local development workflow. Development happens at three levels of integration, each suited to a different class of work. The common case is developing a single component in isolation. Each repository has its own development setup with `cargo run`, `tauri dev`, or hot-reload as appropriate. Components that use the mocks from `os-sdk` need no external dependencies running. Fast feedback loop, runs entirely on the developer's machine.

For shell and compositor work, Smithay supports nested mode: the compositor runs as a regular Wayland window inside the developer's existing desktop. The full shell appears inside that window. No VM or extra hardware is needed. This covers the majority of shell and compositor development; eBPF and kernel-level components are not active in nested mode.

Full system work requires QEMU. This covers eBPF development, the boot sequence, the update system, and anything that requires a real kernel or a complete system environment. The `distro` repository contains a script set for this workflow, including `build-image.sh` for a complete bootable image, `run-vm.sh` for an interactive session, `run-vm.sh –share ~/projects/kernel-layer` to mount a host directory via virtiofs into the VM so that the developer can compile on the host and test in the VM without a full image rebuild, and snapshot management scripts for saving and restoring VM state.

A strict rule applies to eBPF development: eBPF programs are developed and tested exclusively inside the VM, never directly on the development machine. A bad eBPF program can destabilize the kernel; the isolation of the VM is not optional.

Mock strategy. Every IPC interface in the system has an official mock implementation that lives in os-sdk directly alongside the interface definition. Mock and real implementation share the same type definitions. An outdated mock is a build error, not a runtime surprise: if an interface changes, the mock must be updated in the same pull request.

```
#[cfg(test)]
use os_sdk::graph_daemon::mock::GraphDaemonMock as GraphDaemon;

#[cfg(not(test))]
use os_sdk::graph_daemon::client::GraphDaemonClient as GraphDaemon;
```

Listing 17 Conditional mock usage in tests

There are no per-repository custom stubs and no fixture files that can drift from the real schema. The mock is the contract.

Project planning tooling. A project of this scope has cross-cutting concerns that span multiple repositories simultaneously: an Epic about the Spotlight Module API touches desktop-shell, os-sdk, module-sdk, and potentially compositor. Standard per-repository issue tracking is not sufficient for this. The requirements are hierarchical issues or Epics that span multiple repositories, dependency tracking between issues, a roadmap view across all components, and cross-repository milestone coordination.

Three options are under evaluation. The first is GitHub Projects v2 with a meta-repo convention: cross-cutting Epics live as issues in the blueprint repository and component repositories link back to the relevant Epic; GitHub Projects v2 aggregates issues from all repositories into one board. The advantage is zero additional tooling and everything staying in GitHub. The disadvantage is that dependency tracking is honor-system only with no enforcement, there is no proper roadmap view, and Epic-to-child linking is manual.

The second option is Linear, a proprietary hosted tool with strong multi-repo GitHub integration, real Epics with child issues, a dependency graph, and a roadmap view. It has the best user experience for exactly this use case but is proprietary, hosted externally, and costs money at team scale.

The third is Plane, an open-source Linear alternative that can be self-hosted. Feature parity with Linear is growing but not yet complete. Its advantage is that self-hosting fits the project's digital sovereignty philosophy with no external dependency; the disadvantage is additional operational overhead and some features that are still maturing.

The decision is not yet made. The requirement is clear: cross-repository Epic tracking with dependency enforcement and a roadmap view. GitHub Projects alone is likely insufficient for a project of this scope.

Documentation. Two documentation sites are maintained, both self-hosted. The user-facing wiki at wiki.[project].dev is written by hand in the spirit of the Arch Wiki: prose that explains intent and reasoning, not only what a setting does. It covers concepts, how-tos, troubleshooting, and architecture explanations. Community contributions are accepted via pull requests reviewed by the core team. The wiki source lives in the blueprint repository alongside the architecture documents.

The developer API reference at api.[project].dev is auto-generated from code comments: rustdoc for Rust components, typedoc for TypeScript components, aggregated and hosted under a single domain so there is one place to search across all components. Keeping it current is a CI gate: a pull request that removes or modifies a public API without updating its documentation comment does not merge.

10.2 Sustainability and Business Model

This is an open source project. The code is and remains open; that is a foundational commitment, not a marketing position. At the same time, building and maintaining a full operating system is a serious engineering effort that requires sustained funding. The goal is not to extract value from users but to find revenue streams that are structurally aligned with the project's values: user sovereignty, transparency, and European digital autonomy.

Principles. Four constraints rule out entire categories of business models upfront. Core OS functionality is not feature-gated behind a paywall; the desktop, the permission system, the Knowledge Graph, and the AI layer are open and available to everyone. There is no advertising, no telemetry sold to third parties, and no data brokering. Venture capital with growth-at-all-costs expectations is incompatible with a project built on user trust and is therefore excluded. Enterprise features are an acceptable source of revenue; making the core worse for non-paying users is not.

Store revenue. The Store's tiered commission model, described in Section 7.5, generates revenue proportional to ecosystem health. The project earns meaningfully only if developers earn well first. At early stage this is negligible. At meaningful adoption it becomes a significant funding stream. The Store handles application sales, theme sales, module distribution, and profile package distribution under a single infrastructure.

Enterprise services. The Signed Profile Package system described in Section 8.2 is already designed with organizations in mind. The .lenv format is open and documented; anyone can author packages. Organizations deploying managed environments at scale will often want professional help: authoring and maintaining custom packages, deployment tooling, update management, and support SLAs. This is a natural B2B offering that does not touch the open source core. The analogy is Red Hat selling support for software that anyone can download for free.

OEM and hardware partnerships. At sufficient maturity and hardware compatibility, small European laptop manufacturers may want to ship the system pre-installed.

The structural alignment is good: the manufacturer gets a differentiated product, users get a well-tested hardware and software combination, and the project gets sustainable revenue from certification and validation work and an ongoing support relationship. This requires a stable release and a polished out-of-box experience first and is a longer-term play. Relevant candidates in the European market include Tuxedo Computers, the PINE64 ecosystem, and potential Framework partnerships.

Support tiers. A support subscription aimed at power users, small businesses, and developers who want guaranteed response times and direct access offers prioritized bug reports, LTS release guarantees with defined support windows, and a direct communication channel with the core team. No features are locked behind this; it is a service layer on top of the open product, priced modestly enough to be accessible to individual developers.

Public funding and grants. The European open source and digital sovereignty funding landscape is underutilized by most projects. The Sovereign Tech Fund in Germany funds open source infrastructure projects and explicitly values work that reduces dependency on US technology monopolies. The NLnet Foundation funds internet and open source projects with a privacy and user rights focus and has supported numerous small to medium projects. The NGI (Next Generation Internet) programme runs multiple EU funding calls per year for open source, privacy, and decentralization work.

The European digital autonomy angle is not only a philosophical position; it is a genuine funding argument. A European open source operating system built on sovereignty principles is precisely what these programmes exist to support. Grants do not provide recurring revenue but they can fund specific development phases without creating investor obligations or governance compromises.

Path to incorporation. A company makes sense when there is external money to manage, not before. Premature incorporation adds administrative overhead with no corresponding benefit. A realistic sequence: the project gains traction and a community forms; grant applications are filed; Store revenue starts, providing the first external money flow; the first enterprise inquiry for deployment support arrives as the first B2B signal. At that point incorporation becomes appropriate, probably as a GmbH in Austria or Germany, or as a European cooperative structure if community governance is a priority.

The company's role is to employ core contributors, manage the Store infrastructure, and sell enterprise and support services. The open source project remains governed independently of the company: contribution policy, release decisions, and core architecture are not dictated by commercial interests. This separation must be written down explicitly before the company is incorporated, not negotiated afterward under time pressure.

10.3 Roadmap

The roadmap is intentionally sparse at this stage. Defining detailed phase scope before the foundational components are proven would produce a plan that reflects assumptions rather than reality. Phases 1 through the public release will be scoped as Phase 0 concludes and the first components are built and measured.

Phase 0 - Foundation (current). Architecture and design decisions are documented in this blueprint. The proof of concept consists of the Event Bus and Knowledge Graph running as a daemon and collecting basic system events. The base distribution decision between Fedora and OpenSUSE Slowroll is pending and will be made before Phase 1 begins. The Kuzu benchmark described in Section 4.5 at 5,000 events per second is a Phase 0 exit criterion.

Phase 1 and beyond. Phase scope will be defined once Phase 0 is complete. The ordering of subsequent phases follows the architectural dependency graph: kernel layer and event infrastructure before the graph, the graph before the AI layer, the compositor before the shell, the shell before Store and module infrastructure. Public release comes after a stable, tested combination of all layers exists.